

ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE



ROBOTIQUE, 2025-26

Master thesis

DASH-01 :
RL Framework to Develop a Bipedal
Humanoid Robot Aimed at Breaking the
Speed Record

REHAssist

13.04.2025 - 04.09.2025

Author:

Nathann MORAND
Robotics M.Sc.

Supervisors:

Dr. Mohamed BOURI

Abstract

DASH-01 is a 15 kg biped with six quasi-direct-drive actuators, a parallel 4-bar knee and a passive ankle. It was built at REHAssist to answer one question: how fast can a cheap custom biped run? This thesis, although RL oriented in the title, takes the platform from a partially finished mechanical design to a modelled machine that can be simulated and for which RL policy can be trained against. Finally, this thesis assesses the performance envelope of the platform and the policy.

The work covers the whole chain. A CAD-to-MJCF pipeline closes the kinematic loop that URDF cannot express. A measurement campaign to model the plant parameter. A first-principles feasibility sweep bounds the running speed the mechanism admits.

The curriculum has been shown to converge to a running policy within literature reported training steps.

On this platform the binding constraint is the plant rather than the algorithm. The thesis therefore ends in a ranked set of mechanical changes rather than in a controller.

Table of Contents

Abstract	2
1 Introduction and context	5
1.1 The platform and the objective	5
1.2 Change over the year about the objective	5
1.3 Contributions	5
2 Background and literature review	7
2.1 Definitions	7
2.2 Abbreviations	7
2.3 Bipedal locomotion approach	7
2.4 Reinforcement learning for locomotion	9
3 The robot and its digital twin	12
3.1 Dash-01	12
3.2 Selecting hardware	13
3.3 The digital twin	15
3.4 Plant identification	17
3.5 Capability: the speed ceiling before any controller	20
3.6 Deployment stack	22
3.7 Hardware experiments	24
4 Reinforcement learning: attempts and architecture	26
4.1 Common ground	26
4.2 The action-space ladder	27
4.3 Curriculum over base degrees of freedom	32
4.4 The pitch wall	33
4.5 Closing on the measured plant	36
4.6 Negative results and simulation hygiene	39
5 Results	41
5.1 Hardware delivered	41
5.2 Simulated locomotion	41
5.3 Three controlled comparisons	42
5.4 What survives the plant correction	43
5.5 Recorded runs	43
6 Discussion	45
6.1 The distance to the objective	45
6.2 Verdict on the three hypotheses	45
6.3 Ranked levers for a successor robot	45
6.4 Levers for this robot, in the order that unblocks the next	46
6.5 Method-level lessons	47
6.6 What a successor project should do differently	47
7 Conclusion and future work	49
7.1 What was set out to do, and what was delivered	49
7.2 The answer to the research question	49
7.3 Limitations	49

7.4	Future work	49
8	Appendix	51
8.1	From CAD to a simulable model	51
8.2	Component databases	51
8.3	The CAN protocol, as measured	52
8.4	Training configuration	52
8.5	Logbook	53
	References	55

1 Introduction and context

This chapter states the objective, the platform, and the scope. It closes with the contributions.

1.1 The platform and the objective

DASH-01 is a bipedal robot built at REHAssist. It carries two legs, six quasi-direct-drive actuators, a parallel 4-bar knee, a passive ankle spring and a single-board computer. The hardware budget was roughly CHF 5000. Figure 3.1 shows the machine as modelled.

The objective at kickoff was maximum running speed, with the bipedal 100m record as the ambition. That record have moved a few time in the last months and now stands at 8.86s, or about an average of 40.63 km/h.[2] Faster than Usan Bolt’s 9.58s. Before the begining of this thesis the record was held by Cassie in 2022 [6] at 24.73 s for the 100m dash.

The project inherited a partially finished robot. The work reported here therefore starts at modelling, and at the selection of the non-mechanical subsystems. It then runs through characterisation and the training of a policy.

1.2 Change over the year about the objective

The record moved faster than the thesis. New bipedal speed marks now appear at roughly a monthly cadence, from teams with institutional budgets.

The measured plant does not admit record pace. Section 3.5 bounds this mechanism at about 2.7 m/s in burst, and about 1.3 m/s thermally sustained. The bound comes from first principles, by using the workspace of the leg and the impulse it can deliver.

The objective of this thesis therefore restated as a measured demonstrator plus a reusable method. The deliverable is the chain from CAD to a trained policy.

The problem this addresses. Reinforcement learning has produced legged-locomotion results on research platforms. Reproducing them on a new, custom, imperfectly known robot is non trivial. The plant parameters are uncertain. The actuators hide undocumented internal control loops. The sensors are commodity parts. The gap between “reinforcement learning walks in simulation” and “reinforcement learning walks on this robot” is dominated by modelling fidelity, actuator bandwidth and calibration.

Sensing scope. Proprioception and one base inertial measurement unit are the only sensors. Foot impact sensors were dropped due to lack of time rather than on evidence. The hardware provision for them remains in place, as four analogue inputs on the sensor hat. The literature prior that foot feedback helps convergence is therefore untested here.

1.3 Contributions

- **A CAD-to-MJCF pipeline**, with the loop-closure treatment URDF cannot express. The model was then validated against the backdriven workspace (Section 3.3).
- **A measured plant** (Table 3.4). It covers weighed segment masses, a torque constant identified by a known-mass method that breaks the degeneracy between K_t and gravity, the Bode response of the drive’s internal position loop, inertial-sensor noise characteristics, loop timing and the CAN protocol.
- **A first-principles capability analysis** of running speed on that plant, validated against MuJoCo. A mass-distribution sensitivity analysis is also performed to rank the design levers

(Section 3.5).

- **An action-space and curriculum design** for the learned controller. Multiple action spaces were tested, and a curriculum was built to converge to a running policy on the measured plant (Sections 4.2.4 and 4.3).
- **A deployment stack on the robot.** It is a 200 Hz daemon on stock Linux, a web bring-up instrument that was used to perform the measurement of the robot characteristics. The deployment stack also includes safety mechanisms to prevent the robot from exiting the safe operating envelope (Section 3.6).
- **Two component databases**, covering 140 commercial actuators and roughly 430 battery cells. They were used to validate the validity of the choice of the robot part. (Section 3.2, Section 8.2).

2 Background and literature review

This chapter provides a brief context for bipedal locomotion and, more specifically, for the Reinforcement Learning framework used in the literature to address this problem.

2.1 Definitions

First, a few definitions are provided, as the following terms will recur throughout this thesis.

Underactuation is the presence of more degrees of freedom than independently actuated degrees of freedom.

Unilateral contact means that a foot can push against the ground but cannot pull it, and that the resulting contact force must lie within the **friction cone**.

The point at which the resultant contact force acts on the ground is the **centre of pressure**, which is confined to the contact patch.

Duty factor is the fraction of a gait cycle during which a given foot remains in contact with the ground. A **flight phase** is an interval during which neither foot is in contact with the ground.

The **sim-to-real gap** is the performance mismatch observed when transferring a policy from a simulated plant to the physical robot. This mismatch generally arises because the real plant lies outside the distribution of plants and disturbances encountered during training.

In the context of this thesis, **failure** refers to the termination of an episode following body contact with the ground or leaving the measured workspace.

Stability margin, in the context of Reinforcement Learning, is not necessarily expressed as a classical analytical stability margin. In this thesis, robustness is instead assessed through quantities such as the number of surviving episodes and the variance across random seeds.

Plant, borrowed from control theory, refers to the physical system to be controlled, in this case the robot.

2.2 Abbreviations

RL : Reinforcement Learning

CoM : Centre of Mass

ZMP : Zero Moment Point

2.3 Bipedal locomotion approach

Bipedal locomotion can be approached in different ways, depending in particular on the speed of motion and on the assumptions made about contact and dynamics.

The simplest case is a robot with flat feet walking on flat terrain at sufficiently low speed that inertial effects can be neglected. This was the approach used for WABOT-1 in 1973 [26].

In this case, maintaining stability can be reduced to keeping the projection of the CoM inside the support polygon formed by the feet.

As the robot moves faster, inertial effects can no longer be neglected. The relevant quantity is then not simply the projection of the CoM, but the Zero Moment Point (ZMP), which must remain within the support polygon. This approach was used, for example, in ASIMO [27].

Running introduces a flight phase, which is incompatible with the previous approaches that require at least one foot to remain in contact with the ground at all times.

To enable running, Raibert [19] proposed a foot-placement method based on the following relation:

$$x_f = \frac{1}{2} \cdot \dot{x} \cdot T_s + k \cdot (\dot{x} - \dot{x}_d) \quad (2.1)$$

where x_f is the fore-aft foot placement, measured from the hip at touchdown, in metres; $\dot{x}$ is the current forward speed in metres per second; $\dot{x}_d$ is the commanded forward speed, in metres per second; T_s is the stance duration, in seconds; and k is a velocity feedback gain, also expressed in seconds.

The first term corresponds to the neutral point, which produces no net fore-aft impulse. The second term provides velocity feedback: when the robot is moving faster than commanded, the foot is placed further forward, producing a decelerating effect during stance.

This lineage runs through systems such as BigDog [18] and later Atlas.

The underlying idea is that the gait can be stabilised, to a significant extent, by placing the feet at the appropriate location and time.

This also makes it possible to build running robots with point-contact feet, rather than requiring the large flat feet used by earlier approaches.

The approach was subsequently refined by solving the foot-placement and whole-body motion problem online, leading to what is commonly formulated as a Model Predictive Control approach [13].

This approach was central to the development of the early dynamic legged robots developed by Boston Dynamics.

These approaches rely on a sufficiently accurate model of the plant.

In practice, however, the plant parameters are only partially known and may vary with configuration, load, contact conditions, and hardware.

Classical model-based controllers are therefore designed around a particular model and possess a finite robustness margin with respect to modelling errors and disturbances. Beyond this margin, the controller may no longer be able to stabilise the system.

Reinforcement Learning provides an alternative way of obtaining this robustness. Rather than explicitly deriving a controller from a single plant model, RL trains a policy over a distribution of simulated plants. The policy can therefore be exposed to variations in plant parameters during training, for example through domain randomisation, with the objective of improving its robustness to model mismatch.

The problem is further complicated by the unilateral and saturating nature of ground contact. A foot can only push against the ground, not pull it, and the tangential component of the contact force is bounded by the friction cone. Exceeding this bound results in slipping.

A second complication is the timescale of the dynamics. A legged robot can, to a first approximation, be considered as an inverted pendulum with characteristic length h .

In the absence of corrective input, the characteristic timescale of the inverted pendulum is on the order of

$$\tau \sim \sqrt{\frac{h}{g}}, \quad (2.2)$$

which corresponds to approximately 310, ms for a characteristic length of 1, m.

During running, however, contact with the ground may last only a few tens of milliseconds. Within this timescale, the robot must measure the state of the system, estimate the relevant dynamics, compute a corrective command, communicate it to the actuator, and physically produce the required motion.

This places significant requirements on sensing, computation, communication latency, and actuator bandwidth.

The robot considered in this thesis has a measured actuator bandwidth corresponding to a 0.8, Hz pole in servo mode and approximately 12, Hz in MIT mode.

This limitation will become relevant again when discussing the control architecture and the sim-to-real problem later in this thesis.

2.4 Reinforcement learning for locomotion

2.4.1 Actuator control architecture

Several actuator-level control architectures have been used in the literature. The main ones considered here are direct torque control, direct position control, and impedance control.

Direct torque control allows the policy to command the actuator torque directly, giving it the highest level of control authority.

Direct position control allows the policy to command a position target, which is then tracked by a low-level controller.

Impedance control allows the policy to command a desired mechanical behaviour through parameters such as position, stiffness, damping, and, depending on the implementation, torque-related terms.

The latter provides a compromise between control authority and the requirements imposed on the hardware in terms of latency and bandwidth.

Direct torque control provides the policy with the highest level of flexibility, but also imposes demanding communication and control-loop timing requirements.

In simulation, the policy can directly command joint torques at a high frequency [8].

On the physical robot, however, closing such a fast loop requires the policy, communication bus, actuator controller, and state estimator to operate with sufficiently low latency.

In the present hardware architecture, this approach was not practical through the CAN communication bus at the required frequency.

Direct position control substantially reduces these requirements by delegating the low-level actuator dynamics to the motor controller.

Position targets passed to the joint controller are used by several learned locomotion approaches [24, 20].

Two variants were investigated in this thesis: conventional smart-servo position control and the MIT control mode with fixed compliance parameters [11].

Both allow the RL policy to operate at a lower frequency while the actuator controller handles the faster inner control loop.

Impedance control provides an intermediate level of control authority [4].

Instead of directly specifying actuator torque, or being restricted to position targets, the policy can influence the desired mechanical behaviour through position, stiffness, damping, and/or torque-related parameters.

This avoids the need to close the torque loop at high speed over the communication bus while providing the policy with greater freedom than pure position control.

It also exposes more of the actuator dynamics to the policy and can reduce the sim-to-real gap compared with a smart-servo interface when the frequency and parameters of the inner control loop are not fully exposed.

2.4.2 Gait prior

For the purpose of comparing these approaches, the literature can be organised along a second dimension: the amount and location of prior structure imposed on the gait.

A strong prior can be introduced through a **reference trajectory**.

In this case, the policy is primarily trained to track an authored motion. This approach was, for example, explored on the Cassie robot [28].

This strongly restricts the reachable set of motions and can substantially simplify learning.

However, the quality of the resulting behaviour is bounded by the choice of reference trajectory: there is no guarantee that the selected trajectory is optimal for the particular robot morphology and dynamics.

A less restrictive prior can instead be introduced through a **trajectory generator** whose parameters are modulated by the policy [10].

This preserves the assumption that locomotion should have a structured, periodic form while allowing the learned controller to search for a gait adapted to the plant.

Trajectory generators can themselves provide different levels of freedom over the gait parameters.

At the stricter end of this generator-based formulation, the policy only modifies a small number of **high-level gait parameters**, such as phase, frequency, stride length, or amplitude [3].

This is similar to the reference-trajectory approach in the sense that the policy operates within a predefined family of trajectories. However, instead of learning the low-level joint commands required to reproduce a reference gait, the policy learns the high-level parameters that generate it.

This provides a relatively small search space but limits the set of gaits that can be represented.

A more moderate prior allows the policy to modify the **complete trajectory**. Rather than predicting every trajectory sample independently, the possible trajectories can be represented using a low-dimensional basis, such as Fourier series [22] or Bézier curves.

The policy then modifies the coefficients of this representation.

This retains useful structural assumptions such as periodicity and smoothness, while allowing the shape of the gait itself to become a learned variable.

It is also possible to use **neither a trajectory generator nor an explicit gait-shape prior**.

In this case, the policy learns the complete joint trajectory directly [7].

Periodic locomotion can still be encouraged through reward shaping, but periodicity then becomes an emergent property rather than an explicit constraint.

Such an approach provides a larger reachable set, but considerably increases the exploration and learning problem. There is also no guarantee that the learned solution will converge to a periodic gait.

Finally, the prior, when present, can be introduced at different levels: through the **reward** [23], through a **reference motion** [17], or through an authored **trajectory generator** [10, 3].

These choices determine how much of the gait is fixed and how much remains available to the policy at run time, at the expense of increasing or reducing the complexity of the learning problem.

2.4.3 Curriculum design

The third major design dimension is the curriculum used to train the policy.

To help the policy converge, the training environment can be modified to make the task easier at the beginning and gradually increase its difficulty as the policy improves.

The literature has explored a variety of curriculum-design choices, including the order in which different tasks are presented, the rate at which difficulty is increased, and the specific parameters that are modified to create the curriculum.

Terrain and command curricula. A common scheme starts every agent on flat ground with a narrow command range.

As its performance exceeds a predefined threshold, the agent is promoted to rougher terrain or a wider velocity range, and can be demoted otherwise [20].

Difficulty therefore tracks each agent’s competence rather than following a fixed schedule.

This approach is well suited to massively parallel training, where each agent can be promoted or demoted independently.

Automatic domain randomisation. Rather than fixing the domain-randomisation range before training, the range can be widened automatically once the policy exceeds a performance threshold at its current extremes [16].

The randomisation range itself becomes the curriculum parameter, expanding only as fast as the policy can tolerate.

This approach requires a reliable in-training performance signal.

A noisy signal can cause the range to grow faster than the policy can adapt, destabilising training.

Whether the curriculum targets terrain, command range, or randomisation range, an adaptive schedule removes the need to manually tune a progression rate.

In its place, the design requires a performance threshold and a metric with which to evaluate it.

This introduces a problem similar to that encountered in reward design: if the metric does not accurately represent the intended notion of competence, the agent may optimise the metric rather than the desired behaviour.

3 The robot and its digital twin

This thesis needed to first finish the Dash-01 robot. This implied to choose the computer, its actuator and the communication bus. It also needed a battery.

Once that was done, its digital twin was develop from the CAD data and validated against measured value from the assembled robot. Finally the model what could not be derived from the cad was measured on the real robot (like actuator bandwidth) to limit the sim2real risk.

3.1 Dash-01

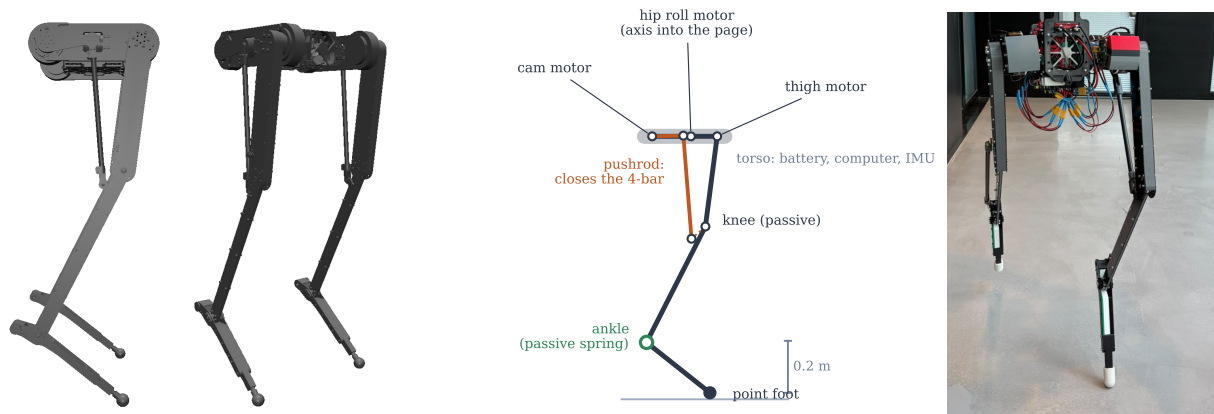


Figure 3.1: DASH-01, from the simulated model and in picture.

	Hip roll	Cam, thigh	Segment	Mass	Count
Part	AK60-39	AKE90-8	Base	5764 g	1
Reduction	39:1	8:1	Hip, with two motors	3271 g	2
Peak torque, datasheet	72 N·m	170 N·m	Cam	66 g	2
Peak torque, measured	61.2 N·m	144.5 N·m	Thigh	483 g	2
Continuous torque	24 N·m	55 N·m	Push rod	71 g	2
No-load speed	10.3 rad/s	22.0 rad/s	Ankle spring assembly	249 g	2
K_t (line-to-line)	0.12	0.272 N·m/A	Shin	324 g	2
R (line-to-line)	0.60	0.164 Ω	Foot	222 g	2
$K_m = K_t/\sqrt{R}$	0.15	0.674	Total	15 136 g	
Reflected armature	0.046	0.0216 kg·m ²			
Mass, unit price	0.75 kg, 393	1.40 kg, EUR 424			

Table 3.1: The six CubeMars quasi-direct-drive actuators (left) and the weighed segment masses (right). Measured peak torque is the datasheet figure scaled by the effective joint K_t of Section 3.4.1. The weighed total supersedes a CAD placeholder of 12 828 g. The robot is therefore 18 % heavier than the model its early policies trained against, and the shin body is $2.6\times$ heavier than modelled. The no-load speeds follow from the 48 V bus and the KV ratings, as $80 \times 48/39$ and $35 \times 48/8$ rpm at the joint.

Each leg carries three motors. Hip roll sits at the base, and a cam and a thigh axis drive the sagittal plane. The knee is passive and follows through the 4-bar. The ankle is passive and follows through a spring (not rendered).

3.2 Selecting hardware

3.2.1 Actuators choice

A database to select the actuator was assembled. Major robotic actuator manufacturers website was scraped for datasheet and a large language model was used to parse each datasheet. The resulting database holds 140 entries across 11 suppliers, listed in Table 3.2.

Manufacturer	Actuator count
ZeroErr	32
MyActuator	29
CubeMars	22
SteadyWin	21
Maxon	12
MAB Robotics	8
RobStride	7
HEBI Robotics	6
Soceboz	1
RSL-ETH	1
mjbots	1
Total	140

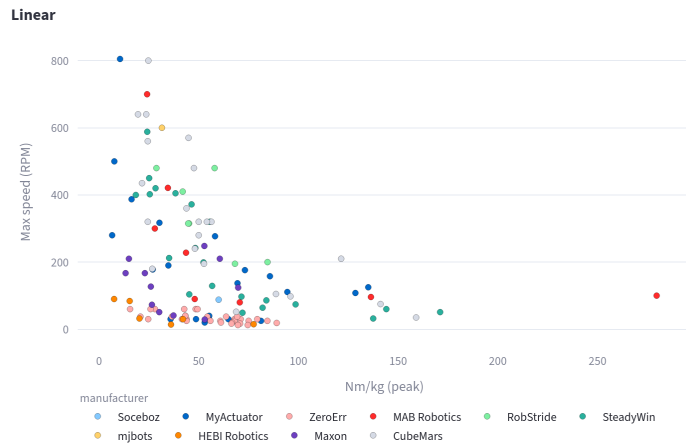


Table 3.2: Actuator manufacturers represented in the database

Figure 3.2: Excerpt of the *motormap* dashboard, showing the actuator database used to select the drive.

The database was compiled into an interactive dashboard, *motormap* [15], built from the *actuatorReview* code [14]. The choice of the actuator itself is detailed in the master’s thesis of M. Aebi [1].

3.2.2 Battery, compute and sensing

The non-mechanical hardware splits into four subsystems: the battery, the compute board, the proprioceptive sensing and the CAN network bus. Table 3.3 summarises them, and the rest of this section derives and measures the entries.

Battery	
Cell	EVE INR21700-50PL, 5.0 Ah
Configuration	13S2P
Nominal voltage, capacity	48.1 V, 10 Ah (468 Wh)
Continuous rating	100 A, cell-limited
BMS limit	200 A
Software cap, burst	250 A, 2 s tier
Compute	
Board	Raspberry Pi 3B+, stock non-real-time Linux
Control loop	200 Hz, 50 % of the frame budget
Sensing	
IMU	ICM-20948, 10-axis, on a Waveshare Sense HAT (B)
IMU interface	I2C, ± 4 g, ± 1000 dps, 200 Hz sampling
Analog input	4-channel ADC on the same HAT
Network bus	
Controller	mcp251xfd over SPI, dual-channel CAN HAT
Topology	two independent 1 Mbit buses, one per leg
Measured load	26 % of one bus

Table 3.3: The robot’s non-mechanical architecture: battery, compute, sensing and the CAN network bus. The battery figures are derived in this section, the control-loop and CAN-load figures are measured in Section 3.4.4.

Six drives on a 48 V bus with short violent current peaks is a sprint duty cycle. The pack is therefore optimised for power density. The choice was again made against a purpose-built database, of roughly 430 cell models, 54 BMS modules and 110 commercial packs (Section 8.2). Its central convention is that *current is a curve, not a number*. Every rating carries a duration stamp, and falls into a continuous, pulse-at- t or fault tier.

The pack built is 13S2P of EVE INR21700-50PL: 48.1 V nominal, 10 Ah, 468 Wh and 1.87 kg of cells, behind a 200 A BMS with a 250 A software cap. The database corrects the specification it was sized against. The 50PL is a 5.0 Ah cell at 50 A continuous and 180 A for 2 s. The pack was sized on a 125 A marketed peak with no published duration. That is exactly the class of number the tier convention exists to reject. The real ratings are 100 A continuous (4.8 kW), 184 A for 10 s and 360 A for 2 s. The continuous tier is *cell*-limited rather than BMS-limited.

The 250 A software cap is therefore a 2-second-tier number, enforced with no duration logic. A policy drawing 250 A for thirty seconds runs the cells at 25 C against a 10 C continuous rating, and nothing notices. The fix is an I^2t accumulator in the control daemon rather than a scalar clamp.

Two survey findings change the verdict rather than decorate it. **Nothing in the commercial 48 V market is built for a sprint duty cycle.** Of 110 packs surveyed, the space holds stationary storage and e-bike packs, and the closest match is rated in the 40 A class. That is roughly $2.5\times$ short, and it is what justifies building a pack at all. Second, on the project’s own criterion the built pack was the wrong pick. High-rate LiPo was rejected as unsourceable, and a Swiss domestic supplier stocks 1S high-rate cells. A 13S1P build of those delivers 481 Wh at 250 A continuous and 500 A for 10 s. The built pack delivers 468 Wh at 100 and 184 A, and the alternative costs 0.67 kg and EUR 463 more. Cycle life, pouch-cell assembly and laboratory storage restrictions are honest defences of the built pack, and none of them was the recorded reason.

Voltage sag is the largest unmeasured quantity on the robot. The drives publish position, speed, current, temperature and an error flag, and **no bus voltage** (Section 8.3). Pack sag under a sprint is therefore not observable from the drive side. A 13S2P arrangement multiplies cell resistance by 6.5, and a high-drain 21700 typically sits at 12 to 15 m Ω . That would put the pack near 80 to 100 m Ω , and imply 15 to 18 V of sag at the ten-second tier. Those figures are an inference from a typical cell rather than a measurement. The reason to record them is that Section 3.5 establishes no-load speed as the binding axis, and no-load speed is linear in bus voltage. A voltmeter across the pack during a sprint is the cheapest outstanding experiment in the project.

Compute and sensing. The board is a **Raspberry Pi 3 Model B**, running stock non-realtime Linux. It was chosen over the laboratory’s usual BeagleBone Black for fluency rather than for merit. An mcp251xfd controller over SPI provides CAN as two independent 1 Mbit buses, one per leg. The Pi also hosts a WiFi access point and the web interface, through which most measurements in Section 3.4 were taken. Section 3.4.4 settles by measurement that this board holds a 200 Hz control loop at half its time budget.

Beyond the motor encoders, the only proprioceptive sensors are the base inertial measurement unit and the motor current estimates. Contact sensors were considered and never fitted (Section 1.2). The unit is an **ICM-20948** on a WaveShare Sense HAT (B), read over I2C at ± 4 g and ± 1000 dps, with a 50 Hz internal low-pass and 200 Hz sampling. Measured at rest it gives 1.8 mg and 0.08 dps RMS noise. The noise density is 218 $\mu\text{g}/\sqrt{\text{Hz}}$ and 0.0096 dps/ $\sqrt{\text{Hz}}$. The gyro turn-on bias is repeatable at [1.03, 0.11, -0.53] dps, and the in-run bias stability is 0.010–0.012 dps. Quantisation sits about $15\times$ below the noise floor on both channels, so widening the ranges to survive footfall impacts costs nothing in effective resolution.

Two properties matter more for control than the noise does. **Latency** from physical event to published value is 10 to 20 ms. It comprises low-pass group delay of 5 to 10 ms, output-rate quantisation, the I2C read and the poll period. Against a 5 ms control period that is two to four whole control steps, and the simulation models it as such. And **the attitude filter dominates**. A 6-axis Madgwick filter at $\beta = 0.06$ rad/s rate-limits its accelerometer correction to about $3.4^\circ/\text{s}$. Published attitude is therefore overwhelmingly gyro-driven, and the accelerometer acts as a slow trim. Yaw is pure integration and drifts, so the gyro zero is captured on a stationary robot each session. The magnetometer is not fused, because it sits centimetres from two brushless motors and its reading moves when they do.

One methodological trap is worth recording. Noise figures taken while the robot swayed on its test rig measure the sway, and nothing in the number says so. That produced a yaw-axis figure roughly $8\times$ too high. It also produced a filter sweep in which a narrower low-pass appeared to measure more noise. Both looked like plausible sensor data. Every published figure now carries a provenance label: measured at rest, motion-independent, datasheet-unverified, or uncharacterised. The uncharacterised list is published rather than guessed at. It holds gyro bias temperature dependence, vibration rectification under footfall impact, cross-axis sensitivity, and clipping at ± 4 g.

3.3 The digital twin

The cam actuator drives a pushrod whose distal end ties to the shin below the knee. Hip, cam, pushrod, shin and thigh therefore close a planar 4-bar (Figure 3.1). URDF is a tree format and cannot express that closure. An exported URDF describes a different robot, one with a floating pushrod, and any simulator consuming it directly produces dynamics that are plausible and wrong. MuJoCo’s MJCF closes the loop with an explicit `<equality><connect>` constraint.

Both files are exported and kept, and only the MJCF is simulated.

The consequence for control is that cam and thigh are *not* independent. The loop assembles over only a diagonal band of the nominal joint box. A target commanded outside that band is physically impossible rather than merely hard to track.

Section 8.1 writes the pipeline as a procedure. Two detours are worth recording. Inertia matrices supplied from a second CAD system arrived in the wrong orientation, and that export never yielded a usable file. The pipeline therefore validates every CAD-supplied inertia tensor against its reference frame explicitly. The second detour is asymmetry. The exported model is not left/right symmetric, and an audit found 17 of 30 mirror-compared quantities off, including a 1.2° rotation in one hip frame. A tool that mirrors the model exactly was built, and deliberately *not* applied. The crooked stance survives symmetrisation, because the gravity settle bifurcates into a lean of about 7° even from a symmetric start. The asymmetry chose only which way. The deployment lineage instead randomises the asymmetry (Section 4.5), because the real robot is asymmetric too.

Validating the twin. Validation is a chain rather than a claim. The constrained toe Jacobian matches `mj_jacGeom` to better than 10^{-10} , and self-tests re-check it on every run. The MJCF closes the 4-bar at 100 % of the poses backdriven on the real robot. That is 1383 of 1383 on the left and 1406 of 1406 on the right, at a residual of 10^{-14} . The test is one-sided, since it would catch only a model band *smaller* than reality. Static equilibrium predictions agree with MuJoCo within 1 %, and the known-mass experiment of Section 3.4.1 ties the model’s gravity torque to a measured current. Every segment was weighed rather than fitted (Table 3.1), which removes those parameters from the identification problem and leaves only the inertia tensors free.

The residual gap is bounded. The CAD centres of mass survive scrutiny (Section 3.4.3), while the link inertia *tensors* remain CAD placeholders scaled by measured mass ratios. That is why the domain-randomisation range on inertia stays at $\pm 25\%$ while the mass ranges narrowed.

3.3.1 Establishing that a simulation result is real

Physics runs in MuJoCo [25] on the CPU, chosen for contact modelling and for not requiring recent GPU hardware. Training nonetheless runs $1.9\times$ faster with the policy network on a GPU, at about 2900 environment-steps per second against 1530 on the same cluster node. Network size is not the cause, since the two-layer MLP costs $117\mu\text{s}$ even on the robot’s Raspberry Pi. The cause is core contention, because PyTorch on the CPU steals cores from the parallel MuJoCo workers. Training ran on the EPFL Izar cluster, which removed throughput as a bottleneck.

A simulation bug does not usually announce itself. It produces a clean, converged, entirely plausible learning curve for a robot that does not exist. Four examples were found while setting up a single ankle study, and each would have silently produced a wrong answer.

- **Joint limits are not rigid by default.** MuJoCo’s default limit softness let body weight push 3.4° past the stop of a supposedly rigid strut. Stiffening the constraint reduced the penetration to 0.08° .
- **A constraint that ships inactive is inactive.** The rigid arm re-settled its stance with a free ankle, because the locking equalities switch on only at episode reset. It initialised 268 mm lower than intended.
- **A safety criterion measured in the wrong frame.** One arm stood outside its own threshold from its reset pose, and every episode terminated 0.1 s in.
- **A stale keyframe.** The shipped stance is an equilibrium of one particular ankle law, so arms

with a softer ankle lurched at every reset.

A fifth class is environmental. A weld constraint that held on MuJoCo 3.7.0 locally gave way on the cluster’s 3.3.7, drifting 0.86 rad over a 60-step rollout. The “rigid” arm was silently compliant on the cluster alone.

The countermeasures are structural. A smoke-test suite builds and briefly runs all 123 training presets before any long launch. Self-tests inside the analysis tools check the analytic Jacobians and the loop closure. Assertions fail loudly when a plant does not match what its preset claims it is. Section 4.6 catalogues the two further artefacts that got through anyway.

3.4 Plant identification

Everything above came from CAD and from datasheets, and so did every training result up to Section 5.2. Between 2026-08-04 and 2026-08-06 those parameters were measured on the assembled robot. Table 3.4 is the summary. No individual row is the finding. Every correction pointing the same way is the finding.

Quantity	Assumed	Measured	Consequence
Total mass	12.83 kg (CAD)	15.136 kg weighed; shin body $2.6\times$	every policy to date trained on a robot that does not exist
Peak joint torque	170 N·m (datasheet)	144.5 N·m ($K_t \approx 85\%$)	simulation $\approx 18\%$ stronger than hardware
Drive position loop	≈ 13 Hz servo	0.8 Hz + 25 ms delay	30 % of commanded amplitude, 94° late, at 2.5 Hz
Ankle spring	$k = 28.65, 12.8$ N·m preload	$k = 41.4$, no preload	145 mm of sag; fails statics at 0.5 BW
Control loop	200 Hz, ms-scale jitter	200.0 Hz, 0.14 ms sd, 50 % of budget	200 Hz confirmed; 50 Hz is measurably worse
CAN load	assumed near the limit	26 % of one 1 Mbit bus	the bus was never the constraint
Ground friction	$\mu \approx 1$	40° slip angle, $\mu = 0.84$	train deliberately at 0.35–0.60

Table 3.4: Assumed against measured plant parameters. Every row moved in the pessimistic direction, and the drive row is the largest single correction, at a factor of 16.

Mass, and how a correction should be applied. The weighed masses were baked into the model. The inertia tensors were scaled by the per-body mass ratio, and the stance keyframe was re-settled under the new load. The decision worth recording is that the correction was *not* made an experimental variable. An earlier arrangement kept a CAD plant and a measured plant as two selectable model files. Every preset that had not explicitly opted in therefore kept training on the 12.83 kg robot, and that was all of them but two. Splitting the plant in two had made the wrong one the default. There is one robot, it weighs 15.14 kg, and the model file says so. The accepted cost is that warm-starting from a pre-correction checkpoint is now a cross-plant transfer.

3.4.1 Peak torque is what the gearbox delivers, not what the datasheet quotes

K_t was measured by clamping a known 1.552 kg mass to the shin. A quasi-static sweep was then repeated with the mass and without it. Fitting the two runs *together* with an indicator column

makes the mass term’s coefficient exactly $1/K_t$. A known mass on a known kinematic lever arm involves no link centre of mass anywhere. That breaks a degeneracy which had defeated the identification until then, since K_t and the gravity-model gain are otherwise confounded and only their product is observable.

The effective joint K_t came out at 82 to 89 % of datasheet, consistent with a gearbox at roughly 85 % efficiency. Peak torque scales with K_t at the same peak current. The cam and thigh actuators therefore moved from 170 to 144.5 N·m, and hip roll from 72 to 61.2 N·m. The second figure is the same efficiency *transferred* rather than measured, because no datasheet K_t is published for that part. It is probably optimistic, because a 39:1 reduction needs more gear stages than an 8:1.

The remaining uncertainty is geometric. The mass was clamped by hand, and K_t trades against its position at about 1.9 % per centimetre. The perpendicular standoff is known only to 5 to 10 cm, and across that box K_t spans 82 to 98 % of datasheet. Settling the figure needs the mass on the shin centreline at a measured distance, not more analysis of the existing data.

3.4.2 The drive’s position loop is 0.8 Hz

The drive’s internal position loop is the single largest modelling error found in the project. It stayed invisible until a swept sine was run on the assembled robot.

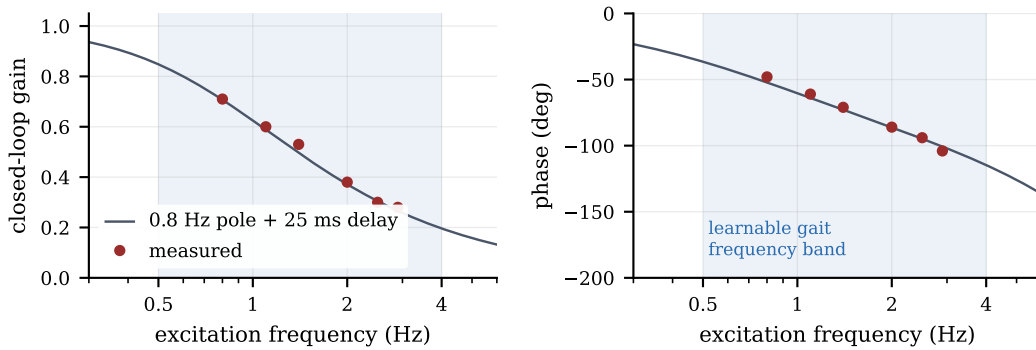


Figure 3.3: Measured closed-loop response of the drive’s internal position loop, identical on both legs to two decimals, against a single-pole model. A gain of 0.71 at -48° is a first-order corner, so one pole at 0.8 Hz plus about 25 ms of transport delay fits the whole sweep. The shaded band is the gait-frequency range the policy can select. At the top of that band the hardware executes under a third of the commanded amplitude, more than 90° late.

The simulation had modelled these actuators as position servos with $k_p = 200$ and $k_v = 5$, critically damped at roughly 13 Hz. The real drive rolls off at 0.8 Hz. The bandwidth mismatch is more than an order of magnitude, against a learnable gait frequency whose top is 4 Hz. At 2.5 Hz the hardware executes 30 % of the commanded amplitude, 94° late. **Every policy in Table 5.1 assumes authority the hardware cannot deliver.**

Three consequences follow. First, the unexecuted part of a command *is* the tracking error, and it tends to the full commanded amplitude as frequency rises. Any high-frequency content therefore caps usable excitation amplitude near 15° , whatever the workspace allows. That cap is why an early identification sweep tripped the safety limit six seconds in. Second, **inertia cannot be excited in position mode at all**, so the quasi-static runs are the valuable data and always were.

Third, the fix is architectural rather than a gain change. The cascaded position loop is proportional-only, and nothing in the runtime can write its gains. The measurement points at

current-mode commands, with the position PD closed on the Pi at 200 Hz. It also points at a torque-source actuator in simulation, carrying a real torque–speed envelope. Section 4.2.1 records why that partially reverses a decision taken at the start of the project.

An existing policy cannot be adapted by switching the drive model on. Applying 0.8 Hz in one step to a converged walking policy collapsed it, with nothing else changed. Episode length fell from 1448 to 251, and return from +511 to −78. The drive model is therefore ramped in as a curriculum, logarithmically in frequency, because the equivalent filter coefficient is strongly nonlinear in bandwidth.

3.4.3 System identification, including what it could not resolve

Inverse dynamics through the closed loop was rejected. The 4-bar is near-singular over part of the operating region, and the resulting inverse blows up. Identification is instead a per-joint linear regression of measured current against four terms: a gravity-torque model, viscous and Coulomb friction, an inertia term and an intercept. A constrained Jacobian differentiates the loop closure out.

Four estimator bugs each produced a confident wrong answer. A missing sign convention on the current column made every K_t negative. An unmirrored joint mapping on the right leg left a loop residual of 0.78 m instead of 10^{-8} , reproducibly across three recordings. Pooling rows from joints a given excitation did not move flipped the sign of K_t , and so did omitting the intercept. Both flip it because the drive’s DC current offset then lands in the gravity column. And over-regularising the loop Jacobian inflated the fitted gravity gain by 20 %. With those fixed, the two legs agree to 5 or 6 % rather than disagreeing in sign. That cross-validation is what says the fixes are right.

The result is two things rather than one. The first is a measured, repeatable, per-joint gravity gain. The second is an explicit statement of what the experiment cannot resolve. With K_t pinned by the known-mass test, the model still under-predicts gravity torque by a factor of 1.3 to 1.75. The obvious explanation is wrong CAD centres of mass, and it was tested and **rejected**. Gravity torque is linear in each body’s centre of mass, so the test is a single linear solve. With all 18 parameters free, no regularisation and no plausibility constraint, moving mass explains at most 11 to 19 % of the residual. Ten of the eighteen directions are pure null space, which is where physically absurd fitted offsets come from. Per-body mass, load-dependent stiction and base tilt were rejected on the same grounds.

One test rules out friction of any origin. Friction reverses sign with the direction of travel. Fitting the gravity gain separately on up-strokes and down-strokes puts both on the same side of unity, for all six joints. What remains lies outside the span of every rigid-body inertial parameter. That points at the kinematic model rather than at the mass distribution, and it is unresolved. Knowing which parameters the experiment could not resolve is what stops the next person fitting the same null space again.

3.4.4 Loop timing, and the drives as they behave on the bus

The whole control loop was benchmarked on the robot with the web interface running. That is the realistic case, because the interface consumes roughly 70 % of a core.

Three results follow, and all three contradict earlier assumptions. **The drives broadcast their state at exactly 200.0 Hz, unprompted.** All six do so, on both buses, free-running with the control software stopped. The sensing ceiling is therefore exactly 200 Hz, and a 50 Hz loop would discard three of every four sensor frames.

Rate	Mean period	Jitter (sd)	p99	Max	Budget used
50 Hz	20.148 ms	2.151 ms	22.41 ms	49.29 ms	23 %
100 Hz	10.000 ms	0.251 ms	10.40 ms	12.28 ms	37 %
200 Hz	5.000 ms	0.141 ms	5.21 ms	7.46 ms	50 %
400 Hz	2.504 ms	0.193 ms	3.24 ms	5.13 ms	73 %

Table 3.5: Measured closed-loop timing on the robot’s Raspberry Pi 3B, with the web interface competing for CPU. One tick drains both CAN buses, runs the policy forward and writes six commands.

200 Hz is clean, and 50 Hz is reproducibly worse. The 50 Hz row is a scheduling artefact of the longer sleep rather than frame loss. The case for dropping to 50 Hz had been compute headroom, and compute is not the constraint. Policy inference costs $117\ \mu\text{s}$, or 2.3 % of a 5 ms budget. **The CAN bus was never near its limit**, at about 26 % of one 1 Mbit bus spread across two.

None of this helps with the real bottleneck. Running the outer loop at 200 Hz cannot compensate for an actuator that executes a quarter of the command 94° late. What the measurement establishes is that the Pi holds ample budget to close the position loop itself, which is the proposed fix. Section 8.3 records the protocol as sniffed rather than as documented.

3.5 Capability: the speed ceiling before any controller

Reinforcement learning demonstrates a lower bound on performance. This section supplies the upper bound that the kinematics, the mass distribution and the actuators impose. Learned results are then judged against physics rather than against hope.

Method. The analysis runs directly on the MuJoCo model, which keeps it consistent with the model used for training. Candidate gaits are parameterised by duty factor D , contact length L_c , ride height h and cadence. For each candidate, a simplified running model gives stride length, flight time, swing duration and ground reaction forces. The leg Jacobian then gives joint torques and velocities through the cycle. A gait counts as feasible only if every joint stays inside the admissible 4-bar assembly region, while respecting the actuator torque, speed and power limits. The parameter space was swept exhaustively. Validation against MuJoCo agrees within 1 % on static equilibrium, and the analytic Jacobian matches `mj_jacGeom` to better than 10^{-10} .

Regime	Top speed	Limiting constraint	Added mass	dv/dm
Continuous operation	1.8 m/s	continuous torque	Uniform scaling	−0.14
Cadence ≤ 6 steps/s	3.7 m/s	cam peak power	Torso payload	−0.11
Cadence ≤ 8 steps/s	4.4 m/s	cam peak power	Entire leg	−0.43
Unconstrained optimum	6.9 m/s	cam peak power	Distal leg only	−0.93
Corrected, burst	≈ 2.7 m/s	cam peak power		
Corrected, sustained	≈ 1.3 m/s	continuous torque		

Table 3.6: Maximum running speed by regime, at a $350\ \text{N}\cdot\text{m}/\text{rad}$ ankle (left), and the sensitivity of that speed to added mass (right). The first four rows are computed on the CAD plant, at 12.83 kg and $170\ \text{N}\cdot\text{m}$. The last two propagate the corrections of Table 3.4 through the sensitivity coefficients. Only the continuous rows are thermally sustainable, and the limit there is the $55\ \text{N}\cdot\text{m}$ continuous rating of the cam actuator.

What limits the result. Swing-leg actuator power limits maximum speed, not kinematic reach. The dominant constraint is the cam actuator during swing. It demands about 878 W, namely 64 N·m at 13.7 rad/s, against a theoretical 935 W. 87 % of feasible gaits are limited at that peak-power corner. Raising actuator no-load speed would therefore improve achievable speed more than raising peak torque would. Stance loading is the second constraint: peak ground reaction forces demand about 167 N·m of the available 170.

Kinematic reach does not limit the result, but it is not free either. The leg spans a theoretical fore-aft workspace of 1.06 m, of which only 0.20 to 0.35 m is usable. Near the edge of the assembly region the 4-bar's transmission ratio deteriorates, and joint velocities rise. A flight phase helps substantially. Reducing duty factor from 0.85 to 0.45 raises predicted speed from 4.6 to 6.9 m/s, for a 1.2 cm vertical excursion of the centre of mass. Lower duty factors demand ground reaction forces above 4.5 body weights, and are infeasible.

Two independent agreements are the checks on this analysis. The unconstrained optimum falls at 12.4 steps/s, which is the same high-cadence solution reinforcement learning discovered repeatedly (Section 4.6.1). That comes from physics, with no learning in it. And a static load analysis reproduces the ankle-stiffness threshold the learning experiments had already shown. The passive spring at 28.65 N·m/rad supports only about half a body weight before collapsing under positive mechanical feedback. About 350 N·m/rad holds two body weights in a stable stance.

The corrected ceiling reclassifies the platform. Peak torque falls from 170 to 144.5 N·m, which is a direct hit on the binding constraint. Distal leg mass rises by roughly 0.70 kg across both legs, at -0.93 (m/s)/kg . Propagating both gives roughly **2.7 m/s credible burst and 1.3 m/s thermally sustained**, against 3.7 and 1.8 on the CAD plant. The learned policy's 2.6 m/s therefore no longer sits comfortably inside the envelope. It sits essentially *at* the burst ceiling, and at twice the sustained figure. A 100 m dash at record pace is about 25 s of continuous running against a continuous torque rating, so the sustained number decides whether a record is reachable. This chassis is a walking and jogging demonstrator.

Limitations, and one correction that runs the other way. The analysis is an optimistic upper bound. It neglects torso attitude stabilisation, lateral dynamics, impact losses and control bandwidth. The last is no longer small, because Section 3.4.2 shows the assumption of an executed trajectory fails well below the cadences assumed here. Inertia tensor *shapes* are still CAD, and Table 3.6 shows the result is most sensitive to exactly that quantity. The corrected figures propagate sensitivity coefficients rather than re-running the sweep. The sweep should be repeated on the corrected plant, with the tooling that now exists in validated form.

One flagged inconsistency runs the optimistic way. The sweep caps the cam at $P \approx \tau_0 \omega_0 / 4 \approx 935 \text{ W}$. That is the envelope of a voltage-limited brushed motor, not of a current-limited field-oriented drive. With R known, back-EMF plus IR exhausts the 48 V bus at 16.6 rad/s at the joint. The true envelope is therefore flat at τ_{pk} up to that corner, and peaks near 2.8 kW against a vendor-annotated 1500 W. The burst figures may be pessimistic by a factor between 1.6 and 3. Two cautions keep that from being good news. The 55 N·m continuous rating sets the sustained figure and does not move. And the 0.8 Hz drive bandwidth caps achievable cadence far below any of these power limits.

3.5.1 Static end-effector force map

The cam actuator reaches the knee through the pushrod 4-bar. The transmission from motor torque to toe force is therefore a strong function of posture, and whether the strong direction is the direction running needs had never been checked. The constrained toe Jacobian differentiates

the loop closure out, so it carries the transmission ratio. Statics against the actuator torque box then gives the feasible force set at every reachable toe position. Three findings follow. All three are statics-only, so they bound capability rather than predict behaviour.

- **The leg stands at 96 % of its reach**, with about 3 mm of further extension available anywhere near the stance position. Everything else follows from sitting that close to the leg-extension singularity.
- **Force there is 19:1 anisotropic**: 16.4 body weights straight down against 0.85 horizontally. Over the running stance region the median tilted capacity recovers to 5.95 BW, because compressed postures rotate their strong axis.
- **The motors are not the binding constraint. The ankle is, by a factor of eight**. Over the stance region the actuators give a median vertical capacity of 4.30 BW, and 77 % of the region clears the 3.5 BW running requirement. The passive ankle gives a median of 0.42 BW, and **0.0 %** of the region clears it.

That last point is a third independent route to the conclusion of Section 4.5.1. It is reached from statics, with no learning and no dynamics in it. The linkage and the motor sizing are adequate to generous. The passive foot spring is the design defect.

3.6 Deployment stack

The policies of Section 4 ultimately have to run on the Raspberry Pi, against real drives, with no simulator to reset. Measurement settled three questions about that, and an incident settled a fourth.

Linux without a real-time kernel suffices. Table 3.5 establishes that a 200 Hz loop holds at half budget with sub-millisecond jitter. One Python-specific detail is load-bearing. CPython hands the interpreter lock between threads at most every `switchinterval` seconds, and the 5 ms default is exactly one control period. The web-interface thread could therefore legally hold the interpreter across a whole tick. Dropping the interval to 0.5 ms took late ticks from roughly 10 % to a few percent.

The steady state was then measured with everything running: the control daemon, the 200 Hz sensor thread, the web interface and the flight recorder. It is **188 Hz against a 200 Hz target, at 3 % late ticks**. An A/B under `systemd` overrides shows where the time goes. The full stack runs at 186.7 Hz, disabling the recorder gives 186.3 Hz, and disabling the sensor thread as well gives 194.8 Hz. The recorder is therefore free, and the sensor thread costs 0.23 ms per tick. The missing 6 % is operating-system scheduling.

A methodological trap, again. The first loop-rate figures taken during bring-up read 60 Hz with 34 % late ticks. They produced a confident, committed diagnosis of an expensive failure path in the CAN layer, and the A/B above shows the suspected path costs nothing. The 60 Hz was the measurement disturbing the measured. While the Pi was being sampled, `scp`, remote compilation, `journalctl` and HTTP polling were all running on its four cores. That is the same failure mode as the sensor noise figures taken on a swaying rig. The number was real, and the attribution was not.

The drive as deployed. Three measured properties dominate the design. The only writable surface is the servo-mode command pair `SET_POS` and `SET_CURRENT`, since the internal cascaded gains are board configuration. The drives report no bus voltage. And **the multi-turn position origin is not stable**. It re-randomises on every power cycle, and the incident below is consistent

with it moving between power cycles as well. Every absolute position command is therefore only as good as a calibration captured earlier in the same session. Nothing in the drive signals when that calibration has silently become wrong.

CAN at runtime. Bandwidth is a non-issue, but running under `systemd` made one failure mode routine. The daemon now starts before motor power is on. With nothing on the bus to acknowledge a frame, socketcan’s ten-frame transmit queue fills within one tick, and every subsequent `send()` raises `ENOBUFS`. On the first boot under `systemd` that exception went unhandled in a background thread, and killed the control daemon during its own preflight. The fix treats a send failure as an event rather than a death. Sends never raise past the CAN layer, and a channel failing ten consecutive sends is retried at 5 Hz until one succeeds. A failing send costs nothing, at 188.9 Hz with the backoff disabled against 188.5 Hz with it enabled, so the backoff is kept for forensics rather than for throughput.

3.6.1 The safety envelope as an instrument

A browser-based interface on the robot is the only path through which the hardware is commanded by hand, and the envelope it enforces is also an experimental instrument. Joint limits are *measured* rather than taken from CAD, by backdriving each joint through its full travel with the motors limp. The 4-bar’s assemblable band is a constraint on a joint *pair*. The recorded region is therefore a box in the foot’s fore-aft and lift coordinates, rather than a set of independent joint ranges. That same box is reused in simulation as a training termination criterion, which makes an entire class of simulator-only exploit impossible rather than merely expensive. It is expressed in the base frame, so by itself it cannot distinguish a collapsed ankle from a folded linkage (Section 3.3.1).

Playback is capped by kinematics rather than by preference. A part failed during a high-cadence playback, and the ceiling was cut from 5 to 2.5 Hz on a derived number. Differentiating the stored waveform gives peak joint speed per unit cadence, which is a property of the trajectory shape. The cam demands 359 deg/s per hertz, and reaches no-load speed at 3.5 Hz. At the 5 Hz previously allowed it was commanded at 143 % of no-load speed; at 2.5 Hz it sits at 71 %. Trip conditions on tracking error, joint range and temperature latch the drives to zero current. The tracking-error trip had to be decoupled from the commanded trajectory, once Section 3.4.2 showed that a large tracking error at speed is expected behaviour rather than a fault.

3.6.2 The incident, the flight recorder and the pre-move guard

On 2026-08-10 a joint destroyed itself on the bench. A zero pose had been captured, and by the time an absolute move was commanded the drives’ raw origin no longer matched it. The drive faithfully executed a position command in the wrong frame. It wound `left.cam` through roughly 1.9 output turns, on a joint with $\pm 86^\circ$ of travel. Investigating it was impossible. The only telemetry was a 4096-sample RAM ring at 20 Hz, and nothing was written to disk. That the investigation was impossible was itself the finding.

A three-tier flight recorder was built in response. Six motors at 200 Hz produce 35 kB/s, or 3 GB/day, which fills an SD card and then destroys it. The recorder therefore splits by rate. Tier A writes every field continuously at 20 Hz, into rotated fixed-width binary segments. Tier B is the black box proper: a 40 s RAM ring of the full 200 Hz state, dumped on any trigger with the pre-trigger history included and a 3 s tail. Tier C is one `fsync`’d JSON line per event, append-only and never deleted. Triggers include every safety trip, mode transition, calibration write, warn-level tracking error below the trip threshold, and any encoder-origin jump. Each dump embeds a content-hashed snapshot of the calibration and drive gains live at that instant,

so a recording is self-describing.

Two constraints shaped it. The control loop must not pay for its own instrumentation, so the 200 Hz thread only appends a tuple to a bounded queue, at about $2\mu\text{s}$. A full queue *drops and counts*, stamping the running drop count into the next accepted record. And the robot must never depend on the recorder, so if the writer thread dies the control loop continues. Time is handled by inversion, because the Pi has no real-time clock. A heartbeat is `fsync`'d every 8 s, so the last heartbeat before a gap *is* the power-off time.

The pre-move guard closes the incident's mechanism, a full-authority slew against a stale calibration. Four checks do it, and none adds a step to the normal workflow. The first command after enabling is “hold the raw position you are reporting right now”, for 0.35 s. It never consults the calibration, so it is safe against a completely wrong zero. The raw encoder pose is recorded at every zero capture, and compared before leaving the limp state. Within a live session a 200 Hz watchdog looks for the signature of an origin rewrite, which moves the reported position while the drive reports no speed. Homing uses an 8° tracking threshold rather than the 25° playback value. And a per-joint travel budget stops a guided move that sweeps more than $1.3\times$ the joint's own range without arriving. On 2026-08-10 that check alone would have cut the excursion from 1.9 output turns to under 0.6. A hardware-free test reproduces the incident against a mock bus, with an `origin_shift` fault injection.

What it found in its first hour. The recorder's first boot captured its own deployment failure, the `ENOBUFFS` crash above, which had previously been invisible. Its steady-state telemetry then established that the left-leg bus was electrically dead, with zero frames transmitted and all three motors silent. That turned “the robot feels wrong” into a timestamped fact. It also ended a longer-standing blindness, because nothing in the stack had ever measured the control loop's actual rate. A loop running at a third of its nominal frequency had been indistinguishable from a healthy one.

3.7 Hardware experiments

Everything in this section ran on the physical robot, through the web instrument.

In-air gait demonstration and teach-and-replay. The first hardware milestone was a hand-authored straight-walking gait, with the body clamped in the air. Its safer successor is teach-and-replay. The motors are held limp by streaming zero current, and a leg is backdriven by hand through the desired cycle. A pure-NumPy pipeline then resamples, phase-aligns, detects the left/right mirror, closes the loop with an FFT low-pass and replays the result. Replay exposed a compliance problem that later measurement explained. At low proportional gain the leg drew only about 2 A against a 20 A allowance, and moved with stick-slip judder. That was an early unquantified encounter with the drive-bandwidth ceiling.

Workspace calibration corrected the model in one way. The cam is a full crank. The CAD-guessed range of $\pm 86^\circ$ is wrong. Recorded sweeps span about 245° of cam rotation, and the 4-bar assembles at every angle. The recorded polygon is eroded, and then does triple duty as training constraint, runtime watchdog and offline deploy gate.

Foot arc geometry, measured late and decisive. The stance leg is a fixed-length pendulum held at 99.5% of its reach, so every foothold lies on a single arc: forward reach 31 cm, *backward reach 2.7 cm*. The centre of mass sits at the rear edge of the workspace rather than at its centre, which is why a backward fall can never be caught. Neither the ankle strut nor the pushrod changes this. But 5 cm of crouch takes backward reach from 2.7 to 20.2 cm, for $2\text{ N}\cdot\text{m}$ out of 144.5. Re-solving the balanced keyframe on that basis took the scripted controller of Section 4.4.1 from 0.91 s to 24.7 s at the pitch-free milestone.

The robot's state as this is written. The right leg is fully functional under the 200 Hz stack, measured at 188 Hz. The left leg awaits repair of the joint lost on 2026-08-10.

4 Reinforcement learning: attempts and architecture

Section 3 supplied a measured plant and a speed ceiling. This chapter develops the controller against them. It is organised as a ladder of control authority, from a raw torque command to a structured gait generator wrapped in reflexes. That ladder is also the ablation the thesis wants. Every rung used the same algorithm on the same simulator, so the comparisons are about the *action representation* rather than about the optimiser.

The chapter hands Section 5 five things. A working architecture, a curriculum, a measured verdict on each rung, a classical baseline, and a failure catalogue. The baseline exists to test whether learning was ever the bottleneck. Its answer is the chapter’s central negative result. A learned controller and a 21-gain scripted one fail at the same milestone, at the same time. The reason contains no controller term.

4.1 Common ground

Formulation. The task is a single dash: stand at the line, run, stop. The learning algorithm was held fixed at PPO [21] from Stable-Baselines3, with an actor–critic MLP on CPU-parallel MuJoCo environments. Episodes are capped at 60s. The control rate is 200 Hz, which Section 3.4.4 establishes as both the sensing ceiling and the deployable rate. A milestone is roughly 600 M environment steps and a multi-day cluster job, and the project as a whole spent roughly 3 G steps. That budget is large rather than asymptotic, and the distinction becomes load-bearing in Section 4.4.2.

The toy problem came first. Committing a 12-DOF robot to an unproven pipeline is expensive. The complete chain was therefore exercised once at minimum stakes, on a single-motor inverted pendulum. That chain is simulate, train, export, deploy on embedded Linux, and close a loop over CAN. The observation is deliberately the hardware telemetry and nothing else: position in turns, velocity in turns per second, and torque, exactly as the drive reports them. The export bakes the observation normalisation statistics *into* the ONNX graph. The policy then runs on a Raspberry Pi under `onnxruntime`, closing the loop over CAN at 50 Hz with a watchdog.

The transfer succeeded, and it bought three things a year before the biped needed them. A debugged train-to-embedded-inference path. The discipline of building observations from raw telemetry rather than from simulator conveniences. And early evidence on the compute question of Section 3.2.2.

The reward is the specification, and the optimiser satisfies the specification rather than the intent. That is not an assertion here. Section 4.6 is the evidence.

One property of the reward integral fixes the design of the sprint objective. **A per-step speed reward cannot price time.** Integrating $w v_x$ over a fixed course gives $w \cdot \text{distance}$ whatever the pace, so a fast run and a slow one score identically. What prices time is a constant per-step clock cost, which integrates to $-w_t T$. Correspondingly the alive bonus must be exactly zero, because paying per second of existence rewards dawdling. The break-even pace is the ratio of the two, at 0.25 m/s.

Three groups of terms sit around that core. The anti-skate terms are inherited from Section 4.2.2. A Siekmann-style phase-gated contact schedule [23] makes each foot pay for ground contact during its expected swing window, using the action’s own gait clock. The efficiency terms price torque, motor velocity and positive mechanical power, and ramp in over 60 M steps so they cannot smother gait emergence early. As the stance ratio ramps from 0.65 down to 0.42, the two

swing windows begin to overlap. That is the one term which demands a flight phase rather than fast walking.

Evaluation protocol. Six rules govern every number reported below, and each was earned by a specific bug.

1. **Evaluate with training aids forced off.** Any aid that holds the robot up will be leaned on. The in-training curve then looks excellent while the unaided policy gets worse. That deception landed three separate times before this became protocol.
2. **Evaluate on a fixed task across curricula.** Training reward from runs with different curricula is not comparable.
3. **Use at least 20 seeded episodes.** Six episodes gave a $5\times$ spread on fall rate for the *same* policy.
4. **Use paired seeds and survivor counts under bimodality.** Outcomes on a hard plant are bimodal: a policy either falls at about 2s or survives 60s. One early unpaired comparison “found” a $27\times$ sensor-noise effect that does not exist.
5. **Report peak rather than final under a one-way curriculum.** One run peaked at 113M steps and then declined monotonically for 215M more, while clock-driven ramps kept tightening.
6. **Treat curriculum.json as ground truth on restore.** The evaluator rebuilt the environment from the config’s *starting* values rather than the curriculum’s final ones. It read a 340M-step policy at $32\times$ below its true performance. That was the fourth evaluation-restore bug of the project.

One diagnostic generalises beyond this project. **Seed variance is the margin measurement.** A figure of 20.3 ± 21.6 s against 5 of 5 at the episode cap is not a worse mean. It is bimodality, and a learning curve plotting the mean will not show it.

4.2 The action-space ladder

4.2.1 Direct joint torque: never converged, and abandoned on a wrong premise

The first framework commanded joint torque directly, on a simplified biped, because the robot was still being designed. It was abandoned for two reasons, and the two failures are of different kinds.

It did not converge. The training history from that period is an unbroken sequence of reward-shaping attempts. The fall penalty rose because the robot learned to belly-flop. The alive bonus changed because it learned to dawdle. The feet-air-time thresholds were retuned because it learned to vibrate in place. The underlying problem is that a raw torque action space forces the policy to synthesise two things from scratch at every control step. One is the gravity compensation that keeps the robot standing. The other is the periodic structure that makes it walk. The reward landscape between those two is dominated by ways of falling over that convince the reward the robot is doing well.

It was judged undeployable, on estimates rather than measurements. Commanding torque means bypassing the drive’s internal loop, and streaming current setpoints over CAN from Python on a Raspberry Pi. The timing budget was estimated, and the estimate was pessimistic on three counts. It assumed the achievable loop rate would be low and jittery. It assumed the bus could not carry the traffic. And it assumed that a missed deadline in torque mode leaves the drive holding the last commanded torque blindly, which on a loaded leg integrates into a

runaway rather than a pause.

The measurement campaign reverses two of the three premises. The Pi holds a full 200 Hz loop at 0.141 ms jitter, with half its budget spare, and 50 Hz is reproducibly *worse* (Table 3.5). The bus sits at 26 % of one line, and the drives already broadcast at 200.0 Hz whether asked or not. Only the third premise still holds.

The two things assumed to be the constraint both hold ample headroom. The actual bottleneck lies inside the servo mode that was kept instead, whose position loop rolls off at 0.8 Hz (Section 3.4.2). The current direction is therefore `SET_CURRENT` commands, with the position PD closed on the Pi at 200 Hz, and a torque-source actuator in simulation. That is not a full return to this rung. The policy would still emit position targets, and only the PD loop that closes them would move from the drive to the Pi, where its bandwidth is known and its gains are writable. What is reversed is the premise, not the action space.

4.2.2 Joint position targets: converged to standing, and skated

The second design made the action six position targets, one per actuated joint. The knee and ankle follow passively. In simulation these drive MuJoCo `<position>` actuators, with gains and force ranges from the datasheets. On hardware the action maps one to one onto what the drives accept.

What worked. Standing and balancing converged, after substantial reward shaping. The contact model was switched to a point foot, in which only a 2.5 cm sphere at each toe collides. A policy therefore cannot rest a shin on the ground and damage the real robot. That switch surfaced a floor-penetration exploit. With only the toe sphere colliding, the non-colliding foot and shin meshes could pass *through* the floor. The first apparently perfect balancing policy was largely sinking its centre of mass below the floor into a stable equilibrium. The fix terminates on an upper-hemisphere toe contact, or on any foot or shin vertex below the floor plane.

What failed. Walking did not converge. After 32 M steps the policy reached a healthy-looking reward curve while *sliding* rather than stepping. Both feet were grounded on 57 to 66 % of steps, at a duty factor of 0.74 to 0.83. The median swing lasted one control step. In-contact toe slip of 0.11 to 0.24 m/s accounted for essentially all of the translation, and mean forward velocity was -0.09 m/s under a $+0.5$ m/s command.

Three causes were found and fixed. Nothing priced foot motion while in contact, so a slip penalty was added, with a grounded predicate that survives the micro-hop and ghost-drag dodges. The air-time term could only ever punish stepping, because credit was paid at touchdown and never lifting scored zero. It was made one-sided and capped, then paired with a per-foot stance-time cap and a swing-clearance term. And `condim=3` silently disables torsional and rolling friction, which turns the toe spheres into ball casters and makes skating free.

Two reward-plumbing bugs compounded the problem. Observation normalisation was also normalising the *reward*, dividing a 200-point fall penalty by a return standard deviation of about 110, so it reached the optimiser as -1.8 . And the entropy coefficient on a clipped Gaussian policy amounted to free entropy. The action standard deviation ratcheted up until training and deterministic evaluation were two different controllers.

Why the design was abandoned. The fundamental problem survived every fix. A per-step position policy must rediscover the periodicity of a gait at every control step, because nothing in the representation says locomotion is cyclic. A plant change compounded that. Updating the six motor masses, which total 7.1 kg and more than half the robot, moved the plant far enough that the earlier standing policy no longer transferred. Rather than retune the same representation against a heavier plant, I changed the representation.

4.2.3 Impedance: designed, and not run

The rung between position and torque is impedance, and it is the one the measurements point at. The design is recorded here because the work was done. The status is stated plainly: **it was never trained, because the interface never existed on the hardware.**

The controller is

$$\tau = K_p(\varphi) (q_d - q) + K_d(\varphi) (\dot{q}_d - \dot{q}), \quad (4.1)$$

with the gains a function of gait phase φ rather than constants. Here q_d and $\dot{q}_d$ are the generator's joint target and its analytic derivative, both in radians and radians per second. It maps directly onto the command frame $[q_d, \dot{q}_d, K_p, K_d, \tau_{ff}]$ that the Cheetah lineage established [12, 11]. That frame gives torque-control properties at a slow outer loop, without the servo-mode ceiling.

Three levels of freedom were specified, in increasing cost. A fixed nominal profile. A scalar pair of scalings (α_p, α_d) emitted by the policy. And a small Fourier correction $\Delta K_p(\varphi)$ sharing the basis of Equation (4.2). Coupling the impedance to the gait rather than giving it an independent coefficient set is deliberate. Stance wants stiffness and swing wants compliance, and gait phase is already the coordinate that distinguishes them.

Why it was not run is a hardware fact rather than a research decision. The CubeMars drives expose `SET_POS` and `SET_CURRENT` and nothing else (Section 3.6). Switching the robot to current mode with the PD closed on the Pi was not completed inside the schedule. Training an impedance policy against a simulated interface the robot does not have would have produced another set of numbers on a plant that does not exist. That is the failure mode Section 3.4 exists to prevent. Section 6.3 lists it as the first experiment to run once current-mode control lands, and Section 6 records that H2 is therefore only partially answered.

4.2.4 A Fourier gait generator tuned by the policy

The third design puts the periodicity into the action space itself. The policy no longer emits joint targets. It emits the coefficients of a Fourier series describing one gait cycle, and the environment reconstructs joint targets from those coefficients at the current gait phase. The network no longer has to invent periodicity, only to choose which periodic signal it wants. Section 2.4 places this against the published work, and states what is and is not claimed: emitting basis coefficients as the action is not new here.

Formulation. For the two sagittal joints of the left leg, the deviation from the nominal stance posture at gait phase φ is

$$\Delta q_j(\varphi) = A_j \left[w_0 a_0 + \sum_{k=1}^N w_k (a_k \cos k\varphi + b_k \sin k\varphi) \right], \quad w_k \propto \frac{1}{\sqrt{k}}, \quad (4.2)$$

with $N = 3$ harmonics, so seven coefficients per joint. Here A_j is the per-joint amplitude bound in radians, a_k and b_k are the emitted coefficients, and w_k are fixed harmonic weights normalised to sum to one, which bounds the waveform.

The $1/\sqrt{k}$ schedule was chosen deliberately over the more obvious $1/k$. A $1/k$ schedule biases the series so strongly toward its fundamental that every gait it can express is near-sinusoidal, which is to say walking. The push-off of a running stride is impulsive and needs high-harmonic authority. The result is clipped to $\pm A_j$ around the nominal posture. That bound is *relative*, because the nominal posture is validated and assemblable while the coupled 4-bar assembles over only a diagonal band of the joint box.

dims	channel	role
14	Fourier coefficients	cam and thigh gait shape, Equation (4.2); mirrored to the right leg
1	gait frequency	cadence, 0.5–4.0 Hz; the phase integrates it
3	hip-roll reflex gains	k_p , k_d , bias of the lateral balance reflex
6	per-step residuals	direct corrections on the six PD targets, ± 0.08 rad

Table 4.1: The 24-dimensional per-step action. The first 18 dimensions are the gait *specification*, and the last six are the fast-feedback channel. The observation holds 55 values stacked over 5 frames, or 275 in total. It carries motor position, velocity and torque; the gravity direction and gyro rates; base velocity; $\sin \varphi$ and $\cos \varphi$; a task channel with the run/stop flag and the distance remaining; and the previous action.

The phase advances at a frequency that is itself part of the action, so the policy chooses its own cadence. The right leg is not emitted at all. It is constructed as the mirror of the left, half a cycle out of phase, as $\Delta q_{R,j}(\varphi) = -\Delta q_{L,j}(\varphi + \pi)$.

Per-cycle updates were not enough. The initial implementation made one environment step equal one full gait cycle, so an episode held roughly 30 decisions instead of 1000. It was immediately better than the per-step position policy. A hand-crafted elliptical foot path pushed through the same reconstruction walked at 0.357 m/s open-loop, against the position policy’s 0.01 m/s. The first trained policy walked at about 0.5 m/s, with no skating signature.

It was nevertheless slow, and it fell over as soon as pitch was released. A literature review at that point returned one unambiguous constraint. **No published system achieves running with per-cycle-only parameter updates and no intra-cycle feedback.** Every successful periodic parameterisation keeps a per-step path from observation to action.

The same review found a genuine semi-Markov bug in the formulation. Taking the *mean* of per-step rewards over a macro-step, with a fixed discount per macro-step, makes the total return scale with cadence. The effective horizon therefore varied between 4.8 and 28 s, depending on the gait frequency chosen. The review also found two missing observations. Base velocity was absent, even though the reward maximises it. Distance to the line was absent, which left the value function partially observed and made braking unplannable.

Per-step re-parameterisation plus residuals. The current stack keeps the Fourier gait but re-emits the entire specification at every control step. It adds a per-step correction channel on top.

Rewriting the gait specification mid-cycle is legal but priced. A penalty proportional to $\sin^2(\varphi/2)$ bills every change to the specification. A rewrite is therefore free exactly at the cycle boundary and most expensive mid-stride. The residuals are deliberately never billed by that penalty, because they exist to be per-step. A one-off stumble correction should not have to pay for rewriting the whole gait. A separate magnitude penalty keeps the residual from quietly becoming the controller. That pricing is what the nearest published neighbour does not have [5], where a coefficient change and a stumble correction are the same event to the optimiser.

The separation held as a design and only partly as an outcome, flagged here rather than only in the failure catalogue. After the frequency remap of Section 4.6.1, the gait clock settled near 1 Hz while the actual footfall rate stayed near 6 Hz. The cadence the robot walked at was therefore not the cadence the Fourier channel specified. The obvious reading is that the unbilled residual channel was the open fast path. It was tested and refuted: removing the channel entirely made

the chatter worse (Section 4.2.6). Pricing the rewrite is a necessary condition for keeping the gait channel a gait, and demonstrably not a sufficient one.

4.2.5 Reflexes: what is learned and what is hard-wired

A Fourier series over gait phase is feedforward by construction. It is a function of where the robot is in the cycle, not of what the robot is doing. Encoding balance as a phase-indexed trajectory would make it blind. Three feedback channels therefore sit outside the generator.

Lateral reflex: learned gains, fixed structure. The hip-roll joints are not driven by the generator at all. They follow a linear reflex on base roll, $u = k_p \phi_{\text{roll}} + k_d \dot{\phi}_{\text{roll}} + b$. It is applied with opposite joint-space signs left and right, so that both feet move the same physical way. That is the lateral shift of the centre of mass the reflex is for. The three gains are emitted by the policy, so the controller is learned, but only three numbers of it are rather than a free trajectory. Nothing assumes which sign stabilises. The reflex is necessarily inert while roll is railed, across milestones m1 to m3, so those training curves carry no information about it.

Sagittal reflex: hand-tuned, not learned. A fixed PD on base pitch and pitch rate drives a symmetric fore/aft thigh offset, $u_p = -\text{clip}(k_p^\theta g_x + k_d^\theta \dot{\theta} + b^\theta, \pm u_{\text{max}})$. Here $g_x \approx \sin \theta$ is the fore/aft component of the measured gravity direction. When the robot pitches nose-down, both feet shift forward under the falling centre of mass. That is capture-point control in its crudest form, and the gain scales follow that reading: $k_p \approx h_{\text{CoM}}/\ell \approx 1$ and $k_d \approx \sqrt{h/g} \approx 0.3$.

Why this one is *not* learned is the opposite choice from the hip-roll reflex, and it is worth spelling out. It was introduced when the pitch-free milestone was collapsing in 1.5s, and a learned gain has no useful gradient in that regime. The policy has to survive long enough to experience the correction before it can tune it, and it was not surviving.

The gains were set by a grid search on the pitch-free plant. Values of $k_d \geq 0.3$ oscillate through the one-step action delay and the target smoothing filter. An authority clip above 0.35 rad starts fighting the gait. Those two bounds fix $k_p = 2.0$, $k_d = 0.2$ and a clip of 0.25 rad. On a pitch-kick arrest test the reflex reduces the final pitch excursion by a factor of 2.6 to 6.7 in both directions. It also lifts early-retraining survival from about 18 to about 80 control steps.

That survival figure is what it buys. It is a stabilising prior rather than a controller, and it does not fix the plant. The height-free plant cannot stand at zero action at all, because holding a ride height requires an active gait, so it collapses in about 1.3s whatever the pitch does. An early acceptance gate of “zero action stands for 20 seconds” was simply wrong for this plant, and the kick-arrest test replaced it.

Residuals: the fast channel. The six per-step residuals are the third path, and the one the literature is most emphatic about. They let the controller respond to something unmodelled within a single control step, without rewriting the gait. They are kept small, at 0.08 rad, raised to 0.20 rad once the control rate moved to 200 Hz. They are also magnitude-penalised, so that the Fourier prior stays dominant. All three channels are implemented in pure NumPy, in the same module that reconstructs the gait, with no dependency on the simulator. The identical reflex code can therefore run on the Raspberry Pi against a real sensor, rather than being re-implemented at deployment time.

A rejected fourth channel. The ankles are passive springs with no actuator. A fixed PD was implemented to apply a pitch-restoring *torque* at the ankle joints, gated on that foot being grounded, as an emulated actuated ankle. At 40 N·m of authority it destabilised the running gait outright, collapsing episodes to 28 steps. A phase-blind torque fights the gait it is trying to stabilise. At a gentle 12 N·m it neither destabilised anything nor solved anything. The negative result was still informative. It was evidence that ankle authority was the right lever, and it

motivated the passive-stiffness experiments of Section 4.4.

4.2.6 A second generator: explicit oscillator against the Fourier series

The Fourier parameterisation is one member of a family. The other canonical member is the explicit limit-cycle oscillator of the Ijspeert school [9, 3]. The literature gave no basis to prefer one, so the question was measured. Sixteen cluster jobs shared one plant, one reward and one schedule, with only the generator swapped. Figure 5.2(a) shows the cold start on the pitch-free plant.

Building the oscillator arm forced a foot-space formulation with a measured inverse-kinematics lookup table. Constructing that table surfaced four plant facts. From the stance pose, *both* cam and thigh move the toe mostly fore-aft, and either sign of cam *shortens* the near-straight leg. The intuitive “thigh swings, cam lifts” mapping is therefore wrong, and it produces no step. The 4-bar has a folded solution branch that must be filtered out. The inversion must propagate continuity between cells, or it flips branches mid-swing. And reachable foot lift at mid-stride is only about 8 cm, because the leg is near-straight there.

The verdict is a tie, with complementary strengths. The oscillator is more sample-efficient from a cold start on the hardest milestone. It reaches an episode length of 408 control steps against the Fourier arm’s 237, at 133 M steps. The Fourier generator is more reliable across seeds. On the warm-started pitch-free plant it crosses the line on 5 of 5 episodes at 2.45 m/s, against the oscillator’s 3 of 5 at 2.40 m/s.

The single best controller measured in the whole experiment was an oscillator run. It was the only arm of the sixteen that ever executed the stop phase, at 2.9 m/s with a left/right asymmetry of 0.08. Run up the full ladder, the oscillator cleared m1 to m4, with a 100 m dash at m2 and 2.95 m/s at m3. It then walled hard at m5, where roll is freed: episode length 337 against the Fourier ladder’s 3213 at matched budget, with the plateau confirmed at $2.5\times$ budget. Both share the identical three-parameter roll reflex, so the difference lies in the generator, and it points future work at lateral authority. The Fourier generator was kept for the deployment lineage. The oscillator result was used as tiebreak evidence that the action-space *family* is right.

One ablation refutes a hypothesis stated earlier in this chapter. Removing the per-step residual channel, leaving a pure oscillator, produced **stability without locomotion**. It survived 9700 of 12000 steps, but travelled 15 m of 72 at 0.25 m/s, with the worst cadence of all sixteen arms. The residual channel was the suspected fast open path behind the 6 Hz footfall chatter of Section 4.6.1. Removing it made the chatter worse, so that explanation is wrong. It is also the empirical form of the literature’s verdict that a per-step channel is mandatory. No equivalent arm was run on the Fourier generator, so the result constrains the family rather than this exact controller.

4.3 Curriculum over base degrees of freedom

Asking a fresh policy to solve balance and locomotion simultaneously is too much, so the base is released progressively. The model’s free joint was replaced by six explicit scalar joints (x , y , z , roll, pitch, yaw), plus six equality constraints that can be activated individually. Any subset of the base degrees of freedom can therefore be rigidly railed at reset.

Milestone m1 frees forward translation only, with height railed at a randomised ride height drawn from a measured lookup table. m2 adds height, m3 adds pitch, m4 adds lateral translation, m5 adds roll, and m6 is the real plant. Rigid equality constraints were chosen over clamping the state after each step. They produce the correct reaction wrench *during* the step, which keeps the leg torques faithful. The observation and action dimensions do not change across milestones, so every stage warm-starts from the previous stage’s checkpoint. The whole ladder is a continuation

rather than six retrainings.

What the mechanism buys is localisation. The milestone that fails names the degree of freedom the policy cannot control. That is what turned the pitch failure of Section 4.4 from a general training problem into a specific plant question.

Three rules were learned the hard way. *Gate the advance, not just the opening*, because a curriculum that can only advance will ride a policy down. *Never gate a rescue ramp on the competence it is meant to rescue*. One milestone took six restarts, five of them self-inflicted. The terminal bug was a competence gate holding *both* the difficulty ramp and the efficiency-penalty ramp. For a policy that never reached the gate, the efficiency terms stayed pinned at exactly zero for the entire run. A run that looked like “the robot cannot learn this” was actually “the reward is not the one we designed”. And *diff configurations against the last working run before theorising*, because that is what settled it rather than another hypothesis.

Against the state of the art. Published curricula range over command, terrain, disturbance and randomisation width. The recent shift is from fixed hand-scheduled schedules to performance-gated ones, with a difficulty vector rather than a scalar. This curriculum is weaker on every one of those axes. Release is binary rather than continuous. Advance is hand-scheduled rather than performance-gated. There is a single axis rather than a vector.

What it has that the published work does not is that difficulty is applied to the *controllability of the plant* rather than to the difficulty of the environment. A successor design follows directly, and is written as future work rather than as a result. Release the constraints softly, for example through a virtual stabiliser $F_z = -K_z(z - z_0) - D_z\dot{z}$ with $K_z \rightarrow 0$. Introduce a disturbance ramp *before* full release, so the policy cannot learn that a suppressed mode does not exist. Gate the advance on maximum recoverable impulse rather than on step count. And keep randomisation present from stage zero rather than appending it at the end.

4.4 The pitch wall

Everything through m2 trained well. Releasing pitch at m3 produced the hardest failure of the project. Episodes plateaued at roughly one second, which is the passive fall time, and stayed there. What was ruled out is the useful part.

Seven reward and curriculum variants all plateaued at the same episode length, so the reward was not the bottleneck. They covered no flight demand, efficiency off, a low speed cap, more authority, height shaping and competence-gated hardening. A decaying external pitch assist produced crutch dependence. The in-training curve looked excellent, and with the assist forced to zero at evaluation the policy was *worse* than the plateau and drifting backwards. An anti-crutch penalty on the assist torque gave no improvement at low weight, and was worse at high weight. Extra armature on the base pitch degree of freedom, faded out, gave marginal gains that did not survive the fade.

Raising the control rate from 50 to 200 Hz plateaued at about 1 s, and so did increasing residual authority and filter bandwidth. That implicates the plant rather than the control law, because a learned residual gives more foot-placement authority than any fixed heuristic could. The emulated actuated ankle behaved as described in Section 4.2.5. And lowering the centre of mass by 6 to 12 cm was slightly *worse*. When every software lever saturates at the same value, the constraint is not in the software.

The diagnostic that cracked it. Rather than try a ninth lever, I instrumented the failure. A tool rolls out a policy and plots, per foot, the fore/aft offset between the toe at touchdown and the whole-robot centre of mass. The result was unambiguous. Base pitch ramps monotonically

forward into a pure topple, and the feet land 8 to 9 cm *behind* the centre of mass. Only 11 % of touchdowns landed ahead of it, and those fell progressively further behind. The policy had a serviceable forward-running gait, and had never learned the *capture step* that plants the foot ahead of the centre of mass to catch a forward fall.

The fix was a plant change, and the plant could not be built. Stiffening the ankle spring by roughly a factor of twelve, from 28.65 to 350 N·m/rad, breaks the plateau cleanly. A stiffness $k \leq 200$ barely helps, while $k \geq 350$ climbs steeply. Afterwards the same diagnostic reports 67 % of touchdowns landing ahead of the centre of mass, with base pitch pinned near zero.

One implementation detail cost a wasted run. The standing ankle sits at about ± 0.25 rad while the spring reference sits at ± 0.7 rad, so the spring carries about 12.8 N·m of preload at stance. Raising the stiffness naively balloons that preload, until the foot slams to its reference and the robot flips. The reference must shift at the same time, so that $k(q_{\text{stand}} - q_{\text{ref}})$ is preserved.

That result also overturned an earlier recommendation of my own. After the emulated-ankle experiments I had concluded that the robot needed a policy-controlled actuated ankle: a motor, two more action dimensions and a full retrain. The stiffness experiment removes all three. Section 3.5 arrives at the same 350 N·m/rad threshold from statics, with no learning in it.

Then the spring was measured, at $k = 41.4$ N·m/rad with **no preload** (Table 3.4). Neither number matches the model. The second discrepancy matters more than the first, and it is the genuinely useful finding. At the same k , the modelled spring sags 19 mm at stance *with* its preload, and **145 mm without it**. The preload rather than the stiffness was doing $7.6\times$ of the work. The parameter the study spent its compute sweeping was not the parameter that mattered.

Every result in Table 5.1 was therefore obtained on an ankle that does not exist. The balance result stands as a statement about what the *plant* needed. It is not a statement about what this robot does.

4.4.1 A scripted baseline: 21 gains win at m2 and hit the same wall at m3

Late in the project a classical controller was written for the same plant. It answers a question the learning curves cannot. When training stalls, is the learning the bottleneck, or the robot?

It combines a clock-driven foot trajectory through the measured foot inverse kinematics, once-per-swing Raibert and capture-point placement (Equation (2.1)), low-gain pitch and roll trims, and feedforward inversion of the 0.8 Hz drive. It uses 21 hand-set gains and no training. It is injected through the environment’s residual channel, so that every plant feature is byte-identical to the learning runs.

Its structure is itself an argument about the plant. The placement target is latched *once per swing* rather than tracked continuously. A continuous pitch loop would need crossover around 1 Hz *through* a 0.8 Hz actuator pole, and would spend about 60° of phase before acting. Latching at swing entry instead gives the drive a whole swing, about 1.8τ , which delivers about 83 % of the commanded placement by touchdown. The same arithmetic sets the tuned cadence near 1 Hz, because at 2 Hz the drive returns 0.38 of the commanded amplitude.

The answer is sharp in both directions. On the height-free, attitude-railed plant at m2, the scripted controller runs **60 s on all 5 seeds**, where the learned reference achieves 20.3 ± 21.6 s. On the pitch-free plant at m3 it survives **1.0 s on 0 of 5 seeds**, against the learned reference’s 1.15 s. That is the same wall, to the decimal.

g (m/s ²)	9.81	6.00	3.50	2.00
predicted topple time	0.93 s	1.19 s	1.56 s	2.06 s
measured survival	1.00 s	1.21 s	1.59 s	2.25 s

Table 4.2: m3 survival equals the analytic free-fall time across a $5\times$ change in the plant’s time constant, to 3–9 %, from a formula containing no controller term. Nothing else moves the number. Drive bandwidth from 0.8 to 35 Hz, cadence, oracle velocity, capture gain (more hurts), rear-foot support, centre-of-mass lowering and all 21 gains were swept.

The wall is free fall, proved by a formula with no controller in it. Model the robot as a rigid inverted pendulum about the toe. The passive topple time from an initial lean θ_0 is then $t = \sqrt{h/g} \cosh^{-1}(60^\circ/\theta_0)$. Sweeping gravity in simulation gives Table 4.2.

Three mechanisms compose the wall. The stance keyframe starts every episode with the toes 9.1 cm ahead of the centre of mass, a 5.67° backward lean at $t = 0$. Correcting the keyframe buys exactly the free-fall time of the residual lean, and no more. The single toe-sphere contact gives *zero* centre-of-pressure authority at any ankle stiffness, so the ankle at m3 is not weak, it is disconnected. And foot placement is geometrically outrun. The toe hangs about 0.92 m below the centre of mass, so a pitch rate $\dot{\theta}$ sweeps the contact point forward at $L\dot{\theta}$. That sweep was measured to beat the controller’s corrective foot motion 2:1, until the ± 0.30 m fore-aft workspace saturates in under a second.

Escaping m3 therefore needs one of two things. Either a corrected keyframe, plus a foot with real fore-aft extent and an ankle strong enough to use it. Or step-on-demand triggering in place of a free-running gait clock. That is a plant conclusion rather than a controller one, which is what the identical learned and scripted failures imply.

Two further findings transfer beyond the baseline. The stand keyframe’s control vector is not a static equilibrium: the position servo needs about 6 cm more commanded foot extension to hold 15 kg. Learned policies discover that silently, and hand-built controllers must be told. And *feedforward beats feedback through this drive*. A constant extension offset stands for 20 s, where adding ride-height feedback drops it to 7.8 s. A continuous loop closed through a 0.8 Hz pole, against a plant diverging in 0.31 s, destabilises.

A methodological rhyme closes the section. The baseline’s automatic gain tuner was given the objective $t_{\text{alive}} + 3\text{dist}$. It immediately exploited that objective exactly as a policy would, driving at 1.6 m/s against a 0.3 m/s command to farm the distance term on the pitch-railed plant. **Reward exploitation is a property of optimisation against an objective, not of neural networks.**

4.4.2 What the comparison says about stability margin

?? sets the theoretical expectation. The scripted controller lies inside the policy class, so with an exact plant and unbounded training the learned optimum can only match or exceed it. The measurement disagrees at both milestones, and it disagrees *for different reasons at each*. Collapsing the two into a single “learning underperformed” loses the finding.

At m2, a margin existed and the learning did not find it. The plant is stabilisable, and one of the two controllers stabilises it. Three mechanisms account for the gap, and each is a statement about the optimisation rather than about the policy class.

- **The feedforward inverse is representable but not discoverable.** The scripted controller inverts the drive pole explicitly, as $q_{\text{cmd}} = q + \tau\dot{q}$. Nothing prevents a network from expressing

that map, because it is linear in quantities the observation history contains. Expressing it and finding it are different problems. A policy is rewarded for surviving, not for leading the plant by 199 ms, and the credit for a correctly anticipated command arrives several control steps late.

- **Margin is not in the objective, and the variance is the evidence.** The return prices survival time and distance. It prices distance from the failure boundary nowhere, so an optimiser has no reason not to park close to the boundary and collect. The signature is bimodality rather than degradation, which is exactly the measured shape.
- **“Unbounded training” is doing real work in the theoretical claim.** A milestone here is 600 M environment steps, which is large rather than asymptotic.

At m3, no margin existed for either controller. Survival equals the analytic free-fall time of an *uncontrolled* inverted pendulum, to within 3 to 9 %, across a $5\times$ change in the plant’s time constant. With zero centre-of-pressure authority and foot placement outrun 2:1, the set of stabilising action sequences from the initial condition is essentially empty. Every controller drawn from an empty feasible set performs identically. Reinforcement learning optimises over the space of controllers, and it has no purchase on the space of physics. The theoretical margin advantage is not wrong at m3. It is vacuous, because the quantity it promises to enlarge is already zero.

Two observations tie this to the rest of the chapter. The mechanism one would normally use to convert the theoretical advantage into a real one is randomisation over what is not known. On this robot it was measured to cost more than it bought (Section 4.5). The robustness randomisation buys is paid for out of performance margin, and a plant whose margin is already near zero has nothing to pay with. Second, margin is a frequency-domain quantity. Neither method can synthesise phase the actuator does not have, so a 199 ms drive pole against a 311 ms divergence constant bounds what *any* controller can achieve. The claim all of this supports is narrower than either “learning beats classical” or the reverse. **The stability-margin advantage of learning scales with how much margin the plant has to give.**

4.5 Closing on the measured plant

The sprint policies above are not deployable as they stand, independently of the plant correction. They observe two privileged quantities that do not exist on the robot. One is base linear velocity, which would need a state estimator that has not been built. The other is distance remaining to the finish line.

A separate lineage was therefore started, for a drivable demonstrator. It removes both privileged quantities, so velocity must be inferred from a 200 ms strided observation history. It adds per-episode plant randomisation over mass, inertia, friction, servo gains, joint damping, ankle stiffness, gravity tilt and actuation delay. It adds a sensor-noise model, including the accelerometer leakage that makes a real sensor’s “down” swing by several degrees during push-off. And it adds the steering channels the mirror-symmetric gait lacks. It cannot warm-start from the sprint milestones, because both the observation and the action widths changed, so it carries a separate training cost. Its history is a sequence of measured diagnoses.

Steering was broken twice over, and both causes were in the plant model. The first policy could not turn right at all. An open-loop authority test isolated plant from policy, and found two faults. The model is not left/right symmetric: two linkage anchor coordinates differ by about 1 mm in the wrong axes, a CAD-export artefact worth $+0.42$ rad/s of intrinsic yaw. And the two steering channels were scaled inversely to their measured authority. Differential stance width is about $6.6\times$ stronger per unit than differential stride, and the weak channel’s entire authority at 0.26 rad/s was smaller than the intrinsic bias.

The fix randomises the anchor asymmetry by ± 1.5 mm rather than symmetrising the file. The real robot will be asymmetric too, and on a perfectly symmetric plant the policy has no reason to learn the correction that transfers. The yaw-response slope went from 0.19 to 1.22, with both turn directions reachable.

Domain randomisation was the thing breaking the policy, and that was measured rather than assumed. A fixed evaluation over 20 seeded episodes removed one adversity at a time from a trained policy. Pushes changed nothing, at $1.0\times$ the mean time between falls. Sensor noise gave $1.5\times$ and trips gave $1.8\times$. **Plant randomisation gave $19.8\times$:** 95.4 s against a 4.8 s mean time between falls, where a clean plant gives 196 s and zero falls. The policy walked fine. Full-width plant randomisation from step zero was what kept it down, while every other axis already had a curriculum. Randomisation width now rides a gated, retreating ramp like everything else.

The measured plant kills a working policy when applied all at once, and does not when applied as a curriculum. Every correction was activated together: weighed masses, 144.5 N·m torque, the strut ankle, the 0.8 Hz drive with 25 ms delay, and friction deliberately below the measured $\mu = 0.84$. The previous best walker collapsed with nothing else changed, from episode length 1448 and reward +511 to 251 and -78.

Ramping the drive bandwidth logarithmically in hertz over 120 M steps changes the outcome. The filter coefficient is violently nonlinear in bandwidth, so a linear-in-hertz ramp would do almost nothing and then fall off a cliff. The same policy completed the descent and stabilised at **episode length 4938, peaking at 6833, and reward +3271**. That is three times its parent's score, on the real drive, and that checkpoint is the warm-start parent for everything downstream.

The residual gap is bounded and stated. On the hardest configuration the current generation survives a median 9.4 s, with 3 of 12 seeded episodes reaching the 60 s cap, walking at about 0.14 m/s with a limp. That motivated a command-range fix. The command box asked for 0.6 m/s from a plant that tops out at 0.14 m/s, forward commands above 0.3 m/s are fatal, and *backward* is the safest direction.

The lineage also re-proved the ladder. It is the one lineage that trains with all six base degrees of freedom free from step zero. It is also the one that does not converge, showing three plateaus across roughly 1.4 G steps. Meanwhile the only controller to reach 2.55 m/s on the measured plant did so at m3, with roll and yaw railed, in 73 M steps. An audit of the archive covering 97 runs at m3 makes that an empirical claim rather than a preference.

Two bugs from this lineage belong in any practitioner's checklist. A warm-started run inherited the *source checkpoint's* RNG seed. Every warm-started "seed replicate" in the project was therefore a bit-identical duplicate. That was verified across two seeds and 100 rollouts with identical curves, until the seed was re-asserted after loading. The second bug is the evaluation-restore bug behind rule 6 of Section 4.1.

4.5.1 The ankle, closed as a build decision

The 350 N·m/rad result answered a simulation question. The hardware asked a different one. What should be *built*, given that a spring wound to 350 will not be manufactured? Deleting the spring assembly also removes 249 g from each shin, on the axis Table 3.6 prices at -0.93 (m/s)/kg. Three independent arguments closed the question, and Figure 5.2(c) is the learning screen among them.

Statics eliminates the as-built spring before any learning. The easiest load case holds the base and applies 0.5 body weights per foot in double support, against a 3.5 BW running requirement. Even there the as-built spring deflects the ankle 26.6° past flat, and drops the robot

145 mm. It rotates the foot so far that the heel carries 208 N while the toe is driven 11.9 mm *into* the floor at the reset pose. Contact forces stay well behaved throughout, so the collapse is genuine loss of load capacity rather than a solver artefact. No controller fixes that. A policy chooses where to put a foot; it cannot make a spring hold seven times its load.

The compression-capable strut wins the learning screen. Run cold at m3 on the corrected plant, the strut arm reaches an episode length of 2307 control steps. It does so in one third the steps the fully welded ankle needs, at 29 M against 73 M. It also walks $3\times$ more symmetrically. Given time, the welded arm completes the 57 m dash at 2.55 m/s in 5 of 6 episodes. A tension-only strut with no compression capacity manages 0.6 m and stands on one leg. The arms are statically identical otherwise, so compression capacity is the entire story.

And the as-built part fails by arithmetic. The fabricated strut is a solid PETG rod, 500 mm long and 3 mm across. Its slenderness is $L/r = 667$, and its Euler buckling load is **0.24 to 0.33 N**. That is thirty-five times below the 10 N the simulation arm assumed. It is also ten times less moment at the ankle than the foot's own gravity torque, so the rod cannot even hold the foot flat in swing. Its tension capacity of 283 N also falls below the 360 N walking peak. Two parts reproduce the simulated one: a 3 mm pultruded carbon rod at $P_{cr} = 14.1$ N, 4241 N in tension and 5.7 g, or a 6/4 mm carbon tube at 181 N.

The ankle question closed as a build instruction: make it a stiff link, in carbon, and delete the spring mass. Executed, it removes a net 209 g per shin. The shin goes from 573 to 364 g, and the robot to 14.72 kg. Taking 209 g off each shin buys more predicted speed than taking 1.8 kg off the base would.

4.5.2 The rebuilt robot, and a cap that was never there

On 2026-08-10 the hardware caught up with the analysis, and the training plant was re-issued to match. Four things changed. Three are the physical robot converging on the model's recommendations, and one is a modelling gap that had been open the entire project.

The actuators' speed was never capped in simulation. Peak torque had always been limited, at the measured 61.2 and 144.5 N·m. Joint *velocity* was unconstrained, so a position servo could crank a joint arbitrarily fast for free. The record shows what that bought. The celebrated $k = 350$ pitch-balance winner balanced partly by pattering one foot at 11 Hz, with a peak thigh speed of 23 rad/s, *past the motor's 22 rad/s no-load ceiling*. The cap is now set per joint, at the measured no-load output speeds of 10.30 and 22.01 rad/s. It is per joint because the two actuator families differ by $2.1\times$, so any scalar cap is wrong for one of them.

The ankle became the rigid carbon tube of Section 4.5.1, and the mass moved the right way, at a net -209 g per shin. **The drive was retuned from 0.8 to about 3 Hz.** The retune adjusted the boards' position-loop gains with the vendor tool, and was measured by repeating the swept sine. The left thigh moved from 0.86 to 3.35 Hz and the cam from 0.85 to 2.88 Hz. The hip-roll axis was left untouched deliberately, as the in-run control, and moved only from 0.74 to 0.76 Hz. The uniform 3 Hz assumption is therefore flagged. Hip roll uses a different actuator and has not been re-measured, so the roll milestones may need re-baselining.

The bandwidth bracket that justified the retune is Figure 5.2(b). Three identical cold m2 runs used drive bandwidths of 0.8, 3 and 12 Hz, with everything else fixed. At 0.8 Hz, which is the robot as it was, the policy *shuffles and never walks*: duty factor 1.00, then a fall. At 3 Hz it walks, covering 20.75 m at 0.69 m/s with genuine per-foot flight. At 12 Hz, effectively the unlimited drive the whole sprint record was trained in, it is worse than at 3 Hz on gait quality, and folds one leg into the workspace kill. The episode-length curve does not show that last part.

Three conclusions follow. The measured retune sits near the sweet spot. The old unlimited-

bandwidth plant was not merely optimistic but qualitatively misleading. And the check cannot be done by replaying a policy trained at 0.8 Hz at a higher bandwidth. That was tried, and the policy simply moves faster and falls. It has to be trained at the new bandwidth.

On this rebuilt plant the scripted controller was retuned against a full mission schedule: stand, walk forward, stand, walk backward, stand. It completes the mission in one 35 s episode on 3 of 3 seeds, at a drive-delivery ratio of 0.98 against 0.31 on the 0.8 Hz drive. Tuning against the full mission mattered. A constant-velocity tune produced a walker that could not stand, which is the scripted twin of the command-curriculum lesson. **The m3 pitch wall survives the retune.** It is still about 1.0 s, still free fall, for both controller families. That confirms it as structural rather than bandwidth-limited.

A second, more ambitious scripted controller was built on the rebuilt plant, a capture-step state machine. It failed, and on the way down it measured four plant facts that any future controller inherits. At 96 % of reach, commanded leg *extension* does not exist: a -65 mm vertical command moves the toe $+85$ mm forward and 8 mm down through the inverse kinematics. Weight support must therefore be commanded in joint space rather than in task space. Capture steps land on air, because the swung foot hovers 1 to 5 cm high through an entire fall. The cause is rear-workspace inverse-kinematics error plus pitch lift. The 4-bar loop-closure constraint itself yields about 15 cm of ride height under single-leg load, with every actuated joint on target, and only a forward-kinematics height loop including the passive joints can see it. And the plant’s soft modes put a weakly damped vertical body bounce near 1 Hz, exactly where discrete stepping at 2.3 Hz pumps it. That is the architectural reason a shuffling gait beats smarter stepping on this robot.

4.6 Negative results and simulation hygiene

4.6.1 The cadence problem and a cautionary bug

With balance solved in simulation, the remaining pathology was cadence. Policies stepped at about 11 Hz, pattering rather than striding. Hard velocity caps, acceleration caps and a cadence penalty all collapsed the warm-started policy instead of slowing it.

Instrumenting the trained policy’s own action channels found the cause. When the control rate was raised to 200 Hz, the learnable gait-frequency range had been widened to 0.5–50 Hz. The reasoning was that so wide a range would be non-binding. The map from raw action to frequency is linear over that range, so a *neutral* network output of about zero means 25 Hz. The policy left that output near zero and inherited a 24 Hz gait clock. The commanded thigh trajectory reversed sign 35 times a second at ± 0.45 rad, a signal that would destroy the real actuators. Every external limit had been fighting a broken internal default. Remapping to 0.5–4.0 Hz, so that a neutral action means about 2 Hz, produced the best controller of the project. **A parameter range widened to be non-binding can silently become the operating point**, because a network’s default output decodes to the middle of the range.

One pathology survives the fix. After the remap the gait clock slowed to about 1 Hz, while the actual footfall rate stayed near 6 Hz. The policy steps through the fast residual channel rather than through the clock. A ring-down test placed the passive ankle mode at 1.0 Hz rather than 6.3 Hz, so the chatter is a forced oscillation rather than a resonance. The policy pumps its legs at 200 Hz to balance a top-heavy robot. Five levers failed to remove it: a residual-rate penalty, a torque-budget curriculum, a reduced pitch-reflex derivative gain, low-pass filtering of the reflex input, and heavy ankle damping. Every warm-started run stays trapped in the fast-stepping basin. Removing it needs a from-scratch retrain with cadence pressure from step zero, or a plant change such as a lower centre of mass.

What was tried	Outcome	What it rules out
Direct joint-torque learning (Section 4.2.1)	never converged; abandoned on a bandwidth argument later measured wrong	the action space was not the problem, the rejection premise was, and only instrumentation could tell
Per-step position policy for walking (Section 4.2.2)	skated at 32 M steps behind a healthy reward curve	reward curves cannot certify a gait; instrumented gait probes can
Emulated actuated ankle	strong version destabilised the gait; gentle version gave a small non-solving lift	a phase-blind torque fights the gait it stabilises, but the small positive identified the right lever
Decaying external pitch assist, four variants	crutch dependence every time; the unaided policy was worse than with no assist	any aid that holds the robot up will be leaned on, and the in-training curve hides it
Lowering the simulated centre of mass by 6–12 cm	slightly worse	rules out the cheapest candidate plant fix
Hard velocity, acceleration and cadence limits	collapsed the warm-started policy	they were fighting a broken internal default, not the gait
Centre-of-mass identification from excitation runs (Section 3.4.3)	rejected: explains ≤ 11 –19 % of the gravity residual, 10 of 18 directions in the null space	the CAD centres of mass stand, and the experiment cannot resolve them
Per-body mass, load-dependent stiction, base tilt, pivot friction	all rejected as explanations of the same residual	the residual is conservative and gravity-correlated, so no friction model produces it
Pure oscillator without a residual channel (Section 4.2.6)	stability without locomotion: 15 m of 72, worst cadence in the set	refutes the residual channel as the cause of the 6 Hz footfall

Table 4.3: Approaches that produced no usable artefact, and what each one closes off. Each cost real time. In most cases the reason for the failure is more informative than a success would have been.

4.6.2 What did not work, and what each failure rules out

4.6.3 Simulation artefacts that produced clean, converged, wrong results

Section 3.3.1 lists four artefacts found while setting up a single ankle study, plus a solver-version split between the local machine and the cluster. Two more got through and were caught later. The **absent actuator velocity cap** of Section 4.5.2 let a prized policy balance at 23 rad/s on a 22 rad/s motor. And the **gait-frequency remap** of Section 4.6.1 left a comment asserting an unexercised assumption, which stayed wrong for months because nothing in the codebase read it. Six artefacts, each producing a plausible learning curve for a robot that does not exist, is the reason simulation results in this project are given the same scepticism as rewards.

5 Results

This chapter reports what exists, measured. It separates four things: what the hardware delivers, what the policies do in simulation, what the three controlled comparisons say, and which of those results survive the plant correction of Section 3.4. That last separation is the sim-to-real argument, so it gets its own table rather than being left implicit.

5.1 Hardware delivered

The platform is a biped weighed at 15.14 kg, or 14.72 kg after the carbon-ankle rebuild. It carries six quasi-direct-drive actuators (Table 3.1). Its 13S2P pack holds 468 Wh and is rated 100 A continuous and 184 A for 10 s (Section 3.2.2). A Raspberry Pi 3B closes a 200 Hz control loop on stock Linux, at 3 % late ticks with half its budget spare (Table 3.5). Two 1 Mbit CAN buses run at 26 % utilisation. The inertial sensor is characterised to provenance-labelled noise figures, and Table 3.4 tabulates every correction in the measured plant model. The deployment stack runs under `systemd`, and it has diagnosed its first real incident (Section 3.6.2).

The capability analysis on the corrected plant places the mechanism at about **2.7 m/s credible burst and 1.3 m/s thermally sustained** (Table 3.6). The static force map places the actuators at a median 4.30 BW of vertical toe capacity over the running stance region. They clear the 3.5 BW requirement over 77 % of that region. The passive ankle gives a median 0.42 BW, and clears the requirement over 0.0 % of it (Section 3.5.1).

At the time of writing the right leg is fully functional under the 200 Hz stack, measured at 188 Hz. The left leg awaits repair of the joint lost on 2026-08-10. **No trained policy has run on the robot.** That statement is narrower than “nothing has run on hardware”. The hand-authored fixed gait plays on the real motors. The web interface drives all six joints under safety limits. Every measurement in Section 3.4 came off the machine. What has never happened is a learned controller closing a loop on the physical robot.

5.2 Simulated locomotion

The fully free 6-DOF plant runs and does not fall. That is 60 seconds of simulated running, at the episode limit, with zero variance across evaluation episodes, at 2.6 m/s. Two caveats belong next to it. Milestone `m6` balances but runs *through* the finish line without executing the stop, which is a task-completion defect rather than a balance one. And `m5` still topples on a fraction of episodes.

Milestone	Free base DOF	Steps	Survival	Outcome
<code>m2</code>	x, z	180 M	53 s	runs; the reliable working regime
<code>m3</code>	+ pitch	600 M	39 s	completes the dash and stops, 2.6 m/s
<code>m4</code>	+ y	600 M	38 s	completes the dash and stops
<code>m5</code>	+ roll	600 M	26 s	high variance: the weak link
<code>m6</code>	all six	600 M	60 s	never falls; runs through the line
<code>m7</code>	+ pitch, cadence fix	163 M	37 s	best controller: 2.71 m/s, low variance

Table 5.1: Deterministic evaluation of the milestone ladder. `m7` is not a new degree of freedom. It is the `m3` plant retrained after the gait-frequency fix of Section 4.6.1. Survival is capped at 60 s by the episode limit, so `m6` is at the ceiling. **Plant epoch: CAD masses at 12.83 kg, a $k = 350 \text{ N}\cdot\text{m}/\text{rad}$ ankle with preload, datasheet peak torque, and an idealised drive near 35 Hz.** None of those four match the measured robot; see Table 5.2.

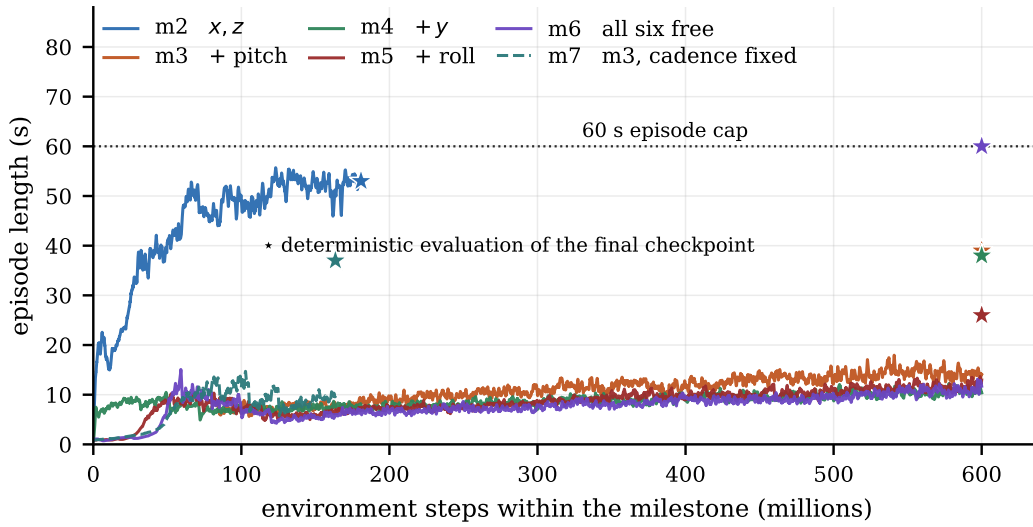


Figure 5.1: The milestone ladder, one curve per rung, each warm-started from the previous rung’s checkpoint. The line is the mean training episode length under the stochastic policy. The star is the deterministic evaluation of that rung’s final checkpoint, which is the number Table 5.1 reports. The gap between the two is exploration noise, and it is why training curves are not used here to certify a controller. Milestone m2 ran at a 50 Hz control rate and the rest at 200 Hz. The conversion from control steps to seconds uses each run’s own decimation.

Against the pre-correction ceiling of Table 3.6, 2.6 m/s is about 70 % of the credible-cadence figure, and already above the thermally sustainable one. On the corrected plant it sits essentially at the burst ceiling, and at twice the sustained figure.

Figure 5.1 shows the shape behind those numbers, and one feature of it is a result in its own right. The training curves for m3 to m6 sit between 8 and 15 s. The deterministic evaluations of the same checkpoints reach 26 to 60 s. The exploration noise that separates them is why rule 1 of Section 4.1 exists. No claim in this thesis rests on a training curve alone.

Action-space verdicts, one line each. Direct torque never converged, and was abandoned on premises that measurement later overturned (Section 4.2.1). Per-step position targets converged to standing but skated (Section 4.2.2). Impedance was designed and never run, because the drive mode it needs was never switched on the hardware (Section 4.2.3). The Fourier gait generator is the design that works (Section 4.2.4). Its reflex and residual channels are necessary rather than decorative, because removing the per-step channel produces stability without locomotion (Section 4.2.6).

5.3 Three controlled comparisons

The pitch wall belongs to the plant. Learning and a 21-gain scripted controller fail identically at about 1 s. That equals the analytic free-topple time across a $5\times$ gravity sweep (Table 4.2). Only a $12\times$ ankle stiffening breaks it in simulation. It survives the hardware drive retune to 3 Hz, which makes it structural rather than bandwidth-limited. On m2, where a margin does exist, the scripted controller runs 60 s on 5 of 5 seeds against the learned reference’s 20.3 ± 21.6 s. It wins on the variance rather than on the mean (Section 4.4.2).

The rebuilt plant walks under command. With the carbon ankle, the 3 Hz drive and velocity caps, the scripted controller demonstrates the first full command-following mission. It

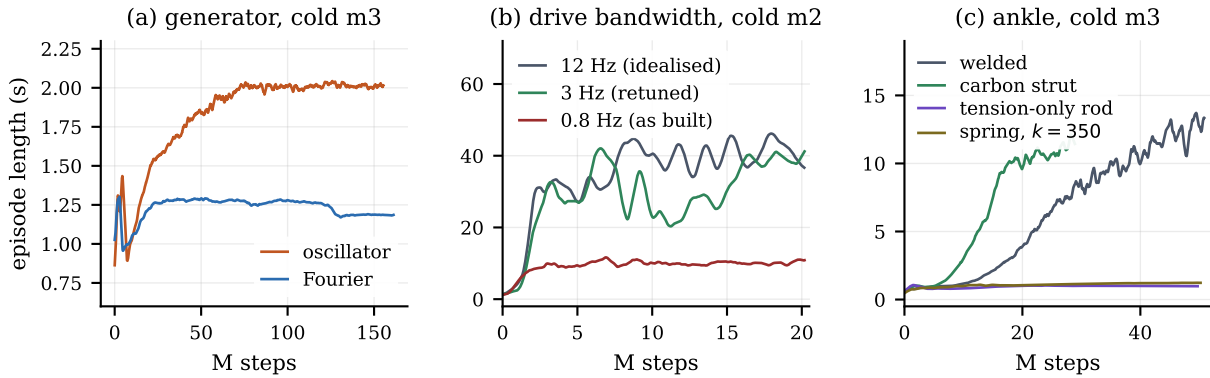


Figure 5.2: Three comparisons, each run with one variable changed and everything else held fixed. **(a)** Generator A/B from a cold start on the pitch-free plant. The oscillator is more sample-efficient cold, at an asymptote near 2.0s against 1.2s. The Fourier arm is the more seed-reliable of the two on warm starts (Section 4.2.6). **(b)** Drive bandwidth on a cold start at m2, truncated to the shortest arm’s budget. The robot as built, at 0.8 Hz, is the outlier and never walks. The retuned 3 Hz plant recovers most of the idealised 12 Hz arm (Section 4.5.2). Episode length does not capture that the 12 Hz arm folds a leg into the workspace kill. **(c)** Ankle variants on a cold start at m3 on the measured plant. The compression-capable carbon strut and the welded ankle climb. The tension-only rod and a spring at $k = 350$ stay pinned at the free-fall time of about 1 s (Section 4.5.1).

stands, walks forward, walks backward and stands again, in one 35 s episode on 3 of 3 seeds, at a drive-delivery ratio of 0.98.

The measured plant is trainable if the drive is ramped. The drivable lineage completes the descent to the real 0.8 Hz drive, and stabilises at an episode length of 4938 and a reward of +3271, three times its parent’s score (Section 4.5). Its diagnosis history is itself a result. It covers the steering asymmetry, the domain-randomisation ablation at $19.8\times$ mean time between falls, and the command-range fit.

5.4 What survives the plant correction

Section 3.4 invalidates the plant every number in Table 5.1 was measured on. Stating which conclusions depend on the plant is more useful than deleting the table, because that distinction *is* the sim-to-real argument.

That pattern was expensive to learn. Roughly 3 billion environment steps of training were spent on a robot 18 % too light and 18 % too strong. Its ankle spring was an order of magnitude too stiff, and its actuator an order of magnitude too fast.

5.5 Recorded runs

Rendered rollouts accompany the results above. Each is named here, so that a claim can be checked against a specific recording rather than against a description. All of them are *simulation*. No footage of a learned controller on the physical robot exists, for the reason given at the head of this chapter.

- m4_stiff_600M, m5_stiff_600M, m6_stiff_600M: the milestone ladder of Table 5.1, greedy evaluation, one seed each.

Survives a plant change	Does not
The action-space comparison: torque, per-step position, and periodic generator with residuals, in that order of tractability.	Every absolute speed and survival time in Table 5.1.
The reward-exploit register, and the finding that reward curves cannot certify a gait.	The claim that the fully free plant runs and does not fall. It ran, on an ankle that will not be built.
The base-DOF curriculum, and warm-starting across stages with fixed observation and action widths.	The 350 N·m/rad ankle-stiffness recommendation, which Section 4.5.1 replaces.
The gait-frequency bug, and its lesson about non-binding parameter ranges.	The 2.6 m/s figure as evidence of hardware capability.
The capture-step diagnostic, and the finding that foot placement lagged the centre of mass.	Any claim of readiness for deployment.

Table 5.2: What the plant correction does and does not invalidate. The pattern is that the methodological results transfer and the performance results do not.

- `m3_CPG`, `m4_CPG`: the oscillator arm of the generator A/B (Section 4.2.6), on identical plants and rewards.
- `walk_fwd_easy_s0`, `teleop_easy_m6`: the drivable lineage on the measured 0.8 Hz drive (Section 4.5).
- `scripted_m2`, `scripted_m3_fail`: the classical baseline succeeding at m2 and hitting the pitch wall at m3 (Section 4.4.1).

Each recording plays at real time unless its filename states otherwise. The plant epoch for each is the one stated in the corresponding table caption.

6 Discussion

This chapter measures the distance between the objective of Section 1.1 and the results of Section 5. It then answers the three hypotheses of ??, and ranks the changes that would close the gap. The ranking is the chapter’s deliverable, and every entry carries the measurement that ranks it.

6.1 The distance to the objective

The target was a bipedal 100 m record pace near 4 m/s. The measured mechanism sustains about 1.3 m/s thermally, and bursts to about 2.7 m/s (Table 3.6). A 100 m dash at record pace is about 25 s of continuous running against a continuous torque rating. The sustained figure therefore decides the record, and the gap is a factor of three.

That gap is mechanical rather than algorithmic, and the evidence is a controlled comparison rather than an inference. A 21-gain scripted controller with no learning in it hits the identical pitch-balance wall at the identical time. The survival time equals the free-topple time of an uncontrolled inverted pendulum, across a $5\times$ change in the plant’s time constant (Table 4.2). At that wall the set of stabilising action sequences is essentially empty, so every controller drawn from it performs identically. No algorithm closes the gap on this plant.

6.2 Verdict on the three hypotheses

H1, dimensionality reduction: supported. The structural prior turned a non-converging problem into a trainable one. The torque attempt is the control. It never converged, across an unbroken sequence of reward-shaping attempts (Section 4.2.1). The per-step position policy converged only to standing, and skated (Section 4.2.2). The Fourier action space walked at 0.5 m/s on its first trained policy, with no skating signature. The claim is scoped to this plant and this sample budget. No arm was run at the 10^8 to 10^9 transitions per condition that the published comparisons use.

H2, control authority: partially supported, and the interesting half is untested. Gait morphology alone was not sufficient. Removing the per-step residual channel from the oscillator generator produced stability without locomotion, at 15 m of a 72 m course (Section 4.2.6). The reflex and residual channels were therefore necessary. The impedance level of the hypothesis, phase-dependent $K_p(\varphi)$ and $K_d(\varphi)$, was designed and never run. The drive mode it needs was never switched on the hardware (Section 4.2.3). H2 is answered for the residual channel, and open for the impedance channel.

H3, robustness across the sim-to-real boundary: untested on hardware. No policy has run on the robot. The closest available evidence is the descent onto the measured 0.8 Hz drive. The structured controller completed it, to an episode length of 4938 and three times its parent’s score (Section 4.5). That is a sim-to-sim result on a measured plant. No unconstrained arm was run through the same descent for comparison, so it does not answer H3 as stated.

6.3 Ranked levers for a successor robot

1. **Bus voltage before torque.** 87 % of feasible gaits are limited at the cam’s peak-power corner (Section 3.5). Over that population, speed scales with no-load speed and is insensitive to peak torque, and no-load speed is linear in bus voltage. The scope is the sweep as run, on the CAD plant. The cost is pack mass plus a drive rated for the higher voltage. A 60 V variant of a drive already surveyed would have given 25 % for EUR 220 (Section 3.2.1).

2. **Distal mass.** At $-0.93 \text{ (m/s)}/\text{kg}$ it carries nine times the leverage of torso mass (Table 3.6). A carbon shin, plus a proximally mounted spring on a tendon, recovers about 0.5 m/s . The cost is a composite part and a tendon route to design. The carbon ankle already executed the cheap half of this, at -209 g per shin (Section 4.5.1).
3. **Re-proportion or replace the 4-bar.** Only 0.20 to 0.35 m of a 1.06 m geometric span is usable. The standing pose sits at 96% of reach, where the toe-force ellipse is $19:1$ anisotropic in the wrong direction (Section 3.5.1). Backward reach from the stance pose is 2.7 cm against 31 cm forward, which is why a backward fall can never be caught (Section 3.7). Target a flat transmission ratio over about 0.5 m , with stance at about 85% of reach. The cost is a linkage redesign. In the meantime, 5 cm of crouch buys most of the backward reach for $2 \text{ N}\cdot\text{m}$ out of 144.5 .
4. **Size the ankle for 3.5 body weights.** The passive ankle delivers a median of 0.42 BW over the running stance region, which is $8\times$ short, and it fails in positive mechanical feedback (Section 3.5.1). The cost is either a stiffer passive element with correct preload, or a seventh and eighth motor. Section 4.4 shows the passive route suffices in simulation, if the stiffness is achievable.
5. **Cool the cam motor.** The demanded $167 \text{ N}\cdot\text{m}$ sits against a $55 \text{ N}\cdot\text{m}$ continuous rating, and the sustained figure is what decides a record. Cooling extends how long the peak stays available. The gain is unquantified, because winding temperature has never been logged at the measured duty cycle. Section 3.5 records that as an open item. This is therefore a lever with a named test rather than a ranked result. The cost is added mass and an airflow path at the joint.
6. **Treat torque-mode control as an architectural requirement from day one.** It was abandoned twice here, once on convergence and once on assumed deployability. The measured 0.8 Hz drive loop points back to it (Section 3.4.2). Unlike the levers above, this one carries no number. In my view it is a design opinion rather than a ranked result.

Two items sit outside the ranking, because they are measurement gaps rather than design changes. Pack voltage sag is unobservable from the drives, and plausibly worth 30% of joint speed at the ten-second current tier (Section 3.2.2). A voltmeter across the pack during a sprint settles it. And the actuator's reference frame for reported current is undocumented, which puts an unquantified risk on every measured torque figure (Section 3.2.1). A clamp meter on a stalled motor settles that one.

6.4 Levers for this robot, in the order that unblocks the next

1. **ONNX export and a Pi inference loop.** Everything else waits on it. The toy problem of Section 4.1 already demonstrated the path at one degree of freedom.
2. **Current-mode control with the position PD closed on the Pi at 200 Hz,** replacing the 0.8 Hz drive loop that every measurement identifies as binding. Table 3.5 shows the budget exists.
3. **The carbon ankle strut,** already fitted, and the corresponding retrain on the rebuilt plant.
4. **A from-scratch retrain with cadence pressure from step zero.** Every warm-started run stays trapped in the fast-stepping basin (Section 4.6.1).
5. **The phase-dependent impedance experiment** of Section 4.2.3. Items 1 and 2 make it possible, and it closes the open half of H2.

6.5 Method-level lessons

The lessons below are the ones that survive a plant change, by the test of Table 5.2.

Optimisers exploit objectives, and neural networks are not the culprit. The policy gamed the reward by skating, penetrating the floor, shuttling and committing suicide by penalty. The scripted baseline’s coordinate-descent tuner gamed its own objective identically. It drove at 1.6 m/s against a 0.3 m/s command, to farm a distance term (Section 4.4.1). On those two cases, in my view every objective needs adversarial review before its optimiser gets it.

The margin advantage of learning rests on three premises, and none was paid in full here. The first is an exact plant, which this was not, since every parameter moved. The second is a reward pricing distance from the failure boundary, which this did not, since it priced survival and distance. The third is an asymptotic budget, which 600 M steps per milestone is not. The generalisable diagnostic is that seed variance is the margin measurement. A figure of 20.3 ± 21.6 s against 5 of 5 at the cap is bimodality rather than a worse mean, and a curve plotting the mean will not show it.

A gait prior reads on two axes, and the useful question is which kind rather than how much. A shape prior fixes the trajectory and exposes its timing, so tuning does not enlarge the reachable set. It is therefore safe only when the morphology has a natural template. A structural prior constrains the class and leaves the member free. DASH-01 separates the two, because its cam turned out to be a full crank with no biological analogue. The evidence that the wider space was genuinely used is the cadence agreement between the learned optimum and the independent physics sweep, both at about 12.4 steps/s (Section 3.5). In my view the two-axis reading is a way of comparing systems usually filed under one label, rather than a measurement.

Every assumed parameter measured later moved pessimistically. Mass by +18 %, torque by –15 %, drive bandwidth by $-16\times$, plus friction and the ankle. Measure before training. And when a measurement lands, apply it as *the* plant rather than as an optional variant, or the wrong default gets trained on (Section 3.4).

Simulation results need the same scepticism as rewards. Six artefacts each produced a clean, converged, wrong result. They were soft joint limits, constraints inactive at settle, frame-relative safety checks, a stale keyframe, a solver-version split between the development machine and the cluster, and an actuator velocity cap that was never present. The last let a policy balance at 23 rad/s on a 22 rad/s motor. Self-tests, smoke gates or measurement caught each one.

Instrumentation found the ceilings, and cost something to build. The measurement instrument found the drive ceiling. The flight recorder found a daemon-killing crash and a dead bus within its first hour (Section 3.6.2). The cost is that the measurement disturbed the measured twice: once with the inertial sensor on a swaying rig, and once with the loop rate under SSH load. The second produced a committed wrong conclusion that only an A/B retracted.

Progressive validation localised the failures. The base-DOF ladder localised failures to single degrees of freedom. The toy problem debugged the deploy chain at one degree of freedom. Both are cheap, and both worked.

6.6 What a successor project should do differently

The validated toe-force map and the capability sweep together make a *design search in simulation* the natural next use of the training cluster. Such a search would range over bus voltage, gear ratio, distal mass, linkage proportions and ankle stiffness. It would search over robots rather than over controllers, for a robot already known to be the constraint. The tooling exists in

validated form. The analysis runs on the same MJCF the policies train against, its Jacobians are self-tested, and its predictions agree with MuJoCo within 1 %.

The sequencing lesson is narrower and cheaper. The measurement campaign that produced Table 3.4 took three days, and it reclassified the project. It ran in month five of nine. Run in month one, it would have changed the actuator purchase, the ankle design and the choice of action space. Roughly 3 billion environment steps would then have been spent on the robot that exists.

7 Conclusion and future work

7.1 What was set out to do, and what was delivered

This thesis set out to apply reinforcement learning to a custom high-speed biped. The platform was inherited as a partially finished mechanical design. Three things were delivered.

A **working method**. It comprises the modelling, training and curriculum stack that produces a 2.6 m/s never-falling runner on the free simulated plant. It also comprises the CAD-to-MJCF pipeline that closes a kinematic loop URDF cannot express. And it comprises the deployment stack that closes a 200 Hz loop on a non-realtime single-board computer.

A **measured robot**. Every load-bearing parameter of DASH-01 is replaced by a measurement, with stated uncertainty and stated un-resolvables. The identification reports a repeatable per-joint gravity gain. It also reports explicitly that the mass distribution is not the explanation for the residual, so the next person does not fit the same null space again.

An **account of the distance between the two**. The measured robot is 18 % heavier and 15 % weaker than the model it trained against, and an order of magnitude slower in actuation. Its corrected ceiling is about 2.7 m/s burst and 1.3 m/s sustained. That reclassifies the platform from record contender to demonstrator, and Section 6.3 ranks the changes that would move it.

7.2 The answer to the research question

?? asked at what level learning should sit, given this robot’s actuator dynamics and available control bandwidth. The answer this work supports is narrow. **On a plant whose actuation bandwidth is an order of magnitude below its own divergence timescale, the level at which learning sits matters less than the level at which the actuator is commanded.**

The evidence is that a learned controller and a 21-gain scripted controller hit the identical pitch wall at the identical time. The wall is predicted by a formula containing no controller term. The learning method held wherever it could be tested, and classical control matched it at the plant’s own limits. That agreement is a consistency check on the stack. It is not evidence that either method won, because the plant bound both.

7.3 Limitations

Recorded throughout, and gathered once here. No policy has run on the hardware, and export plus inference on the Raspberry Pi remain the critical path. The sprint results rest on privileged observations, namely base linear velocity and distance to the line. Identification left the link inertia tensors and the 0.9 N·m gravity residual unresolved, with the kinematic model as the remaining suspect. The actuator model lacks a coupled torque–speed envelope, which makes the simulation optimistic exactly in the high-speed regime the project cares about. The corrected capability figures propagate sensitivity coefficients rather than re-running the feasibility sweep. The impedance action space was designed and never trained. And the physical robot currently stands with one functional leg.

7.4 Future work

For *this* robot the record orders five steps, and Section 6.3 gives them with their dependencies. First, ONNX export and a Pi inference loop. Second, current-mode control with the position PD closed on the Pi. Third, the carbon ankle retrain. Fourth, a from-scratch retrain with cadence

pressure from step zero. Fifth, the phase-dependent impedance experiment that the first two make possible.

For a *successor* robot the ranking is bus voltage, then distal mass, then the 4-bar proportions, then ankle sizing. Motor cooling is a lever with a named test, and torque-mode control is a design opinion. Underneath all of them sits one method-level recommendation. A validated force map and a validated feasibility sweep now exist. The natural next use of the training cluster is therefore a design search over robots, rather than a further search over controllers for a robot already known to be the constraint.

8 Appendix

8.1 From CAD to a simulable model

The procedure below is the one that worked, after the detours recorded in Section 3.3. The repository holds the scripts and the exported files.¹

1. **Re-assemble in one CAD system.** Each limb is exported as STEP with its origin centred on its joint, then assembled and exported to URDF with a plugin. Inertia tensors supplied from a second CAD system are validated against their reference frame explicitly rather than trusted, because that export arrived rotated.
2. **Accept that the URDF is an approximation.** URDF is a tree format, so the pushrod floats and the 4-bar is open. The exported URDF is kept for tooling that needs it, and it is never simulated.
3. **Close the loop in MJCF.** Two sites, one at the pushrod tip and one on the shin, are tied by `<equality><connect>`. The residual at the measured workspace poses is 10^{-14} .
4. **Replace CAD masses with weighed masses,** and rescale each body’s inertia tensor by its own mass ratio. This removes the segment masses from the identification problem entirely.
5. **Re-settle the stance keyframe under gravity** after any plant change. A keyframe is an equilibrium of one particular ankle law, and re-settling is not idempotent: re-settling from an already crouched keyframe lands on the 4-bar’s fold branch.
6. **Gate every change with self-tests.** Analytic Jacobians against central differences, loop closure against the model’s own keyframe, and an assertion that a plant matches what its preset claims it is. A smoke suite builds and briefly runs all 123 training presets before any long launch.

8.2 Component databases

Both databases are published as standalone tools, so their verdicts on this robot’s hardware can be reproduced.

Actuator review.² It holds 140 commercial integrated actuators, 121 motors, 121 gearboxes and 9 standalone drives, as four joined tables. A candidate can therefore be composed rather than only looked up. Modelling follows Lee et al., *How to Model Brushless Electric Motors for the Design of Lightweight Robotic Systems* (arXiv:2310.00080).

Two conventions carry the tool. The motor constant $K_m = K_t/\sqrt{R}$ is stored as the portable figure of merit. Rewinding n turns gives $K_t \propto n$ and $R \propto n^2$, so K_m is invariant to the winding. And every electrical constant carries an explicit reference-frame tag: phase, line-to-line, q-axis or bus, in wye or in delta. Under-determined conversions are flagged rather than guessed. The schema’s first missing field is control-loop bandwidth and gain writability, for the reason given in Section 3.2.1.

Battery review.³ It holds roughly 430 cell models, 54 BMS modules and 110 commercial packs, each row carrying a source URL. A synthesiser builds a pack from cell, BMS and series-parallel count. “What I could build” therefore plots on the same axes as “what I can buy”. Every current rating carries a duration stamp, and falls into a continuous, pulse-at- t or fault tier. The pulse

¹https://github.com/nathmo/running_robot

²<https://github.com/nathmo/actuatorReview>

³<https://github.com/nathmo/batteryReview>

tier interpolates as $I(t) = \sqrt{I_{\text{cont}}^2 + (I_{\text{pulse}}^2 - I_{\text{cont}}^2) t_{\text{pulse}}/t}$. Fields inferred rather than read off a datasheet are written ESTIMATED<n>, so that they still parse and can be filtered out.

8.3 The CAN protocol, as measured

There are two 1 Mbit buses, one per leg, with three motors each. The CAN ids are 104 for hip roll, 105 for cam and 106 for thigh. The arbitration id encodes both node and command, as `id | (command << 8)`.

Exactly two frame types exist per motor. Outbound is the command frame, of which only SET_POS and SET_CURRENT are writable. Inbound is a single feedback frame at command byte 0x29. It free-runs at exactly 200.0 Hz, with a p99 period of 5.03 ms, and uses all eight bytes for position, speed, current, temperature and an error flag.

No bus voltage and no duty cycle are reported. The electrical route to K_t is therefore closed, and pack sag is unobservable from the drive side. Six motors, counting command plus reply, occupy about 26 % of one bus.

8.4 Training configuration

Algorithm and budget. PPO from Stable-Baselines3, with an actor-critic MLP on CPU-parallel MuJoCo environments. The policy network runs on a GPU for core-contention reasons rather than for matrix throughput. Throughput is about 2900 environment-steps per second on one cluster node. The physics step is 1 ms. Control decimation is 5 (200 Hz) for every lineage, except the early sprint milestone at decimation 20 (50 Hz). The episode cap is 60 s.

Action and observation. Table 4.1 gives the 24-dimensional action and the 275-value stacked observation.

Reward groups. A per-step clock cost prices time and the alive bonus is exactly zero, for the reason in Section 4.1; break-even pace 0.25 m/s. Anti-skate terms: foot-slip penalty with a grounded predicate, one-sided capped air time, per-foot stance-time cap, swing clearance. Phase-gated contact schedule with the stance ratio ramping from 0.65 to 0.42. Efficiency terms on torque, motor velocity and positive mechanical power, ramped in over 60 M steps. Gait-specification rewrite priced by $\sin^2(\varphi/2)$; residual magnitude penalised; residual rate not billed by the rewrite penalty.

Curriculum schema. Each run writes a `curriculum.json` recording the live value of every ramped parameter. That file is ground truth on restore. Seven axes are ramped across the lineages: base degree-of-freedom release (m1–m6), stance ratio, efficiency scale, domain-randomisation width, control jitter and drop probability, command range, and drive bandwidth. The last is ramped logarithmically in hertz over 120 M steps.

Domain randomisation, set from measurement where one exists. Ground friction is sampled over 0.35 to 0.60, against a measured $\mu = 0.84$. That is deliberately below the real value, because the guarded failure mode is planting a foot that then slides. Joint homing error is sampled at 5° , against a calibration accurate to 1 or 2° . Linkage anchor asymmetry is sampled at ± 1.5 mm. Link inertia tensors are sampled at ± 25 %, wide precisely because Section 3.4.3 could not resolve the mass distribution.

8.5 Logbook

Timeline reconstructed from the repository’s commit history, with the exception of the measurement campaign (Section 3.4) which was performed in parallel to the training. The logbook is a record of the work, not a tutorial. It is therefore selective and omits the many dead ends, the many failed experiments, and the many commits that were only a few lines of hyper-parameter tuning.

- 2026-04-13** Repository created. First framework: MuJoCo plus Stable-Baselines3 PPO on a generic legged environment.
- 2026-04-20** A dozen reward and hyper-parameter commits in one day. The robot learns to belly-flop, and the fall penalty, forward-speed weight, alive bonus and feet-air-time thresholds are each chased in turn.
- 2026-04-21** Tuning starts going backwards. Decision to attack sim-to-real separately on a toy problem.
- 2026-04-30** Pendulum environment, PPO, ONNX export and a Raspberry Pi runtime: the project’s first complete simulation-to-hardware transfer (Section 4.1).
- 2026-05-21** The generic-robot approach is abandoned. The DASH-01 URDF is exported from CAD and the simulator rewritten around it.
- 2026-05-22** Manual URDF repair: the closed loop of the parallel leg cannot be expressed in URDF, and the export angles are wrong.
- 2026-06-22** First training on the real geometry: a standing policy, converged two days later.
- 2026-07-06** **Anti-skating rebuild.** An instrumented rollout measures what the policy is doing: both feet grounded 57–66 % of steps, median swing one control step, -0.09 m/s of travel under a $+0.5$ m/s command. Three causes fixed at once: contact `condim 3→6`, the six motor masses welded onto their stator bodies ($5.73→12.83$ kg), and a standing keyframe settled under gravity.
- 2026-07-06** Hardware bring-up begins in parallel: motor test scripts, slow sine sweeps, torque-control safety limits.
- 2026-07-09** Workspace and joint-limit calibration on the real leg.
- 2026-07-10** Web interface to drive the robot: per-motor sliders, live leg visualisation, gait recording and playback.
- 2026-07-14** Naming settled to DASH-01. Two structural changes: the base-DOF locking curriculum and the Fourier gait controller.
- 2026-07-21** m3 falls in 1.5 s. Pitch reflex, angular-momentum reward and normalisation rejuvenation added.
- 2026-07-22** The real m3 failure is an entropy-gate deadlock: the action standard deviation sat pinned at maximum for a whole 150 M-step run.
- 2026-07-23** The decaying pitch assist turns into a crutch. The control rate moves from 50 to 200 Hz, with the reward made rate-invariant and every curriculum count rescaled.
- 2026-07-24** A diagnostic proves the m3 policy plants its feet 8 cm behind the centre of mass while falling forward: it never learned the capture step. An evaluator bug that evaluated at 100 m rather than the curriculum distance is found and fixed.
- 2026-07-28** **Gait-frequency bug.** The 200 Hz rescale had widened the learnable range to (0.5, 50) Hz, so a neutral action meant a 25 Hz gait clock. Remapped to (0.5, 4.0).

- 2026-07-29** The fix produces the best m3 controller so far. The residual 6 Hz footfall is traced, wrongly at first, to ankle-spring resonance.
- 2026-07-30** Generator A/B result: no clear winner. The full ladder is relaunched twice, once with the cadence fixes and once on the oscillator generator.
- 2026-08-04** **The measurement campaign opens.** Weighed masses replace the CAD placeholders, 12.83→15.14 kg. The toe's static force capability is mapped over the workspace: 96 % of reach, 19:1 anisotropy, the passive ankle 8× short.
- 2026-08-05** System identification debugged: two sign errors, per-joint refitting, reduced Jacobian damping. Peak joint torque corrected to what the gearbox delivers.
- 2026-08-06** **The ankle question closes by statics.** The real spring deflects 26.6° and drives the heel 11.9 mm through the floor at half body weight. The CAN feedback protocol is measured, and a warm-start bug is found: seed replicates had been silent duplicates.
- 2026-08-09** Commanded speeds are bounded by what the plant can reach. The walker gets the base-DOF ladder it never had.
- 2026-08-10** **The plant becomes the robot as rebuilt.** Per-joint motor velocity caps are enforced for the first time. The ankle becomes a rigid carbon tube and the drive bandwidth is bracketed. On the hardware side, a failure that cost a joint prompts the three-tier flight recorder and the pre-move guard.
- 2026-08-11** First full ladder on the rebuilt plant: twelve jobs, about 23 h wall clock. m2 walks out the full 60 s cap; m5 is a hard wall. A scripted, no-learning walk controller is written as a baseline.
- 2026-08-19** **Foot-shape study: sixteen runs, negative result.** Reaching the answer took three wrong readings of the same data, and the harness's own rule, that a per-arm difference smaller than the within-rung spread is not a result, killed all of them.
- 2026-08-25** **Foot arc geometry measured.** Forward reach 31 cm, backward reach 2.7 cm. Re-solving the balanced keyframe takes the scripted controller at m3 from 0.91 s to 24.7 s.

References

- [1] Max A. Aebi. Dash-01: A High Speed Bipedal Running Robot. From Kinematic Design to Mechanical Implementation. Master’s thesis, EPFL, Lausanne, Switzerland, August 2026. Advisers: Mohamed Bouri, Alain Prenleloup.
- [2] Associated Press. Chinese humanoid robot breaks speed record at world games. AP News, 2026. <https://apnews.com/article/chinese-humanoid-robot-beijing-world-games-record-069b76ee6748154fd67bec39fa927cf6>.
- [3] Guillaume Bellegarda and Auke Ijspeert. CPG-RL: Learning central pattern generators for quadruped locomotion. *IEEE Robotics and Automation Letters*, 2022. arXiv:2211.00458.
- [4] Miroslav Bogdanovic, Majid Khadiv, and Ludovic Righetti. Learning variable impedance control for contact sensitive tasks. *IEEE Robotics and Automation Letters*, 5(4):6129–6136, 2020. arXiv:1907.07500.
- [5] Guillermo A. Castillo, Bowen Weng, Wei Zhang, and Ayonga Hereid. Hybrid zero dynamics inspired feedback control policy design for 3D bipedal locomotion using reinforcement learning. *IEEE International Conference on Robotics and Automation (ICRA)*, 2020. arXiv:1910.01748.
- [6] Devin Crowley, Jeremy Dao, Helei Duan, Kevin Green, Jonathan Hurst, and Alan Fern. Optimizing bipedal locomotion for the 100m dash with comparison to human running. *arXiv preprint*, 2025. arXiv:2508.03070.
- [7] Nicolas Heess, Dhruva TB, Srinivasan Sriram, Jay Lemmon, Josh Merel, Greg Wayne, Yuval Tassa, Tom Erez, Ziyu Wang, S. M. Ali Eslami, Martin Riedmiller, and David Silver. Emergence of locomotion behaviours in rich environments. *arXiv preprint*, 2017. arXiv:1707.02286.
- [8] Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, Dario Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26):eaau5872, 2019.
- [9] Auke Jan Ijspeert. Central pattern generators for locomotion control in animals and robots: a review. *Neural Networks*, 21(4):642–653, 2008.
- [10] Atil Iscen, Ken Caluwaerts, Jie Tan, Tingnan Zhang, Erwin Coumans, Vikas Sindhwani, and Vincent Vanhoucke. Policies modulating trajectory generators. *Conference on Robot Learning (CoRL)*, 2018. arXiv:1910.02812.
- [11] Benjamin Katz, Jared Di Carlo, and Sangbae Kim. Mini Cheetah: A platform for pushing the limits of dynamic quadruped control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 6295–6301, 2019.
- [12] Benjamin G. Katz. A low cost modular actuator for dynamic robots. MSc thesis, Massachusetts Institute of Technology, 2018. Handle 1721.1/118671.
- [13] Scott Kuindersma, Robin Deits, Maurice Fallon, Andrés Valenzuela, Hongkai Dai, Frank Permenter, Twan Koolen, Pat Marion, and Russ Tedrake. Optimization-based locomotion planning, estimation, and control design for the Atlas humanoid robot. *Autonomous Robots*, 40(3):429–455, 2016.
- [14] Nathann Morand. actuatorReview. GitHub repository, 2026. Accessed 2026-08-30.
- [15] Nathann Morand. Motormap: An Interactive Actuator Explorer. Streamlit web application, 2026. Deployed instance of the actuatorReview repository, accessed 2026-08-30.
- [16] OpenAI, Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. Solving Rubik’s Cube with a robot hand. *arXiv preprint*, 2019. arXiv:1910.07113.
- [17] Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne. DeepMimic: Example-guided deep reinforcement learning of physics-based character skills. *ACM Transactions on Graphics (SIGGRAPH)*, 37(4), 2018.

- [18] Marc Raibert, Kevin Blankespoor, Gabriel Nelson, and Rob Playter. BigDog, the rough-terrain quadruped robot. In *IFAC Proceedings Volumes (17th IFAC World Congress)*, volume 41, pages 10822–10825, 2008.
- [19] Marc H. Raibert. *Legged Robots That Balance*. MIT Press, Cambridge, MA, 1986.
- [20] Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Conference on Robot Learning (CoRL)*, pages 91–100, 2021.
- [21] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint*, 2017. arXiv:1707.06347.
- [22] Nima Shafii, Siavash Aslani, Omid Mohamad Nezami, and Saeed Shiry. Evolution of biped walking using truncated Fourier series and particle swarm optimization. In *RoboCup 2009: Robot Soccer World Cup XIII*, volume 5949 of *Lecture Notes in Computer Science*, pages 344–354. Springer, 2010.
- [23] Jonah Siekmann, Yesh Godse, Alan Fern, and Jonathan Hurst. Sim-to-real learning of all common bipedal gaits via periodic reward composition. *IEEE International Conference on Robotics and Automation (ICRA)*, 2021. arXiv:2011.01387.
- [24] Jie Tan, Tingnan Zhang, Erwin Coumans, Atil Iscen, Yunfei Bai, Danijar Hafner, Steven Bohez, and Vincent Vanhoucke. Sim-to-real: Learning agile locomotion for quadruped robots. In *Robotics: Science and Systems (RSS)*, 2018. arXiv:1804.10332.
- [25] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012.
- [26] Waseda University. Robots of Waseda: WABOT-1. YouTube, May 2026. YouTube video, published May 7, 2026.
- [27] WhatIMakeOfThis. Asimo Walking. YouTube, July 2020. YouTube video, published July 16, 2020.
- [28] Zhaoming Xie, Patrick Clary, Jeremy Dao, Pedro Morais, Jonathan Hurst, and Michiel van de Panne. Learning locomotion skills for Cassie: Iterative design and sim-to-real. In *Conference on Robot Learning (CoRL)*, pages 317–329, 2019.